

ADVANCE PYTHON PROGRAMMING

1. Printing on Screen

Theory:

Q1. Introduction to the print() function in Python.

The print() function in Python is a fundamental tool for displaying output to the console or standard output. It's used for debugging, displaying results, and providing feedback to users. Here's a breakdown of its key features and usage:

Basic Usage:

- The simplest form of print() takes one or more arguments and displays them on the console, separated by spaces by default.
- Example:

Python

```
print("Hello, world!")
```

```
print(10, 20, 30)
```

Output:

Hello, world!

10 20 30

Key Parameters:

The `print()` function has several optional parameters that allow you to customize its behavior:

- **sep (separator):**

- Specifies the separator between the arguments.
- The default separator is a space (' ').
- Example:

Python

```
print("apple", "banana", "cherry", sep=", ")
```

Output:

apple, banana, cherry

- **end (end character):**

- Specifies what to print at the end of the line.

- The default end character is a newline ('\n').
- Example:

Python

```
print("Hello", end=" ")
```

```
print("World!")
```

Output:

Hello World!

• **file (file object):**

- Specifies the file object to which the output should be written.
- The default is sys.stdout, which represents the standard output (console).
- This is very useful for writing to files.
- Example:

Python

```
with open("output.txt", "w") as f:
```

```
    print("This is written to a file.", file=f)
```

- **flush (boolean):**

- Specifies whether the output should be flushed (forced to write to the output stream immediately).
- The default is False.
- When set to True, the output is immediately written to the stream, rather than being buffered.
- This is useful for interactive applications or when you need to see the output immediately.

Examples:

- **Printing variables:**

Python

```
name = "Alice"
```

```
age = 30
```

```
print("Name:", name, "Age:", age)
```

Output:

Name: Alice Age: 30

- **Formatted output using f-strings (Python 3.6+):**

Python

```
name = "Bob"
```

```
age = 25
```

```
print(f"Name: {name}, Age: {age}")
```

Output:

Name: Bob, Age: 25

- **Printing multiple lines:**

Python

```
print("Line 1")
```

```
print("Line 2")
```

Output:

Line 1

Line 2

- **Printing without a newline:**

Python

```
print("Progress:", end=" ")
```

```
for i in range(5):  
    print(i, end=" ")
```

Output:

Progress: 0 1 2 3 4

Q2. Formatting outputs using f-strings and `format()`.

Both f-strings and the `.format()` method are powerful tools in Python for formatting output, offering more control and readability than simple string concatenation. Here's a breakdown of each:

1. f-strings (Formatted String Literals):

- Introduced in Python 3.6, f-strings provide a concise and efficient way to embed expressions within string literals.
- They are prefixed with the letter `f` or `F`.
- Expressions within curly braces `{}` are evaluated at runtime and their values are inserted into the string.

Basic Usage:

Python

```
name = "Alice"
```

```
age = 30
```

```
print(f"My name is {name} and I am {age} years  
old.")
```

Output:

My name is Alice and I am 30 years old.

Formatting Specifiers:

- You can use format specifiers within the curly braces to control the output's appearance.
- Example:

Python

```
pi = 3.14159265
```

```
print(f"Pi to two decimal places: {pi:.2f}") #:.2f  
means 2 digits after the decimal point and f  
means float.
```

Output:

Pi to two decimal places: 3.14

- More examples:
 - {value:width}: Specifies the minimum width of the field.
 - {value:.precision}: Specifies the precision for floating-point numbers.
 - {value:type}: Specifies the type (e.g., d for integer, f for float, s for string).
 - {value:fill>width}: Right align, add fill characters.
 - {value:fill<width}: left align, add fill characters.
 - {value:fill^width}: center align, add fill characters.

Python

```
num = 42
```

```
print(f"{num:05d}") #output 00042
```

Output:

00042

2. .format() Method:

- The .format() method is available in all Python versions.
- It uses curly braces {} as placeholders within the string.
- The values to be inserted are passed as arguments to the .format() method.

Basic Usage:

Python

```
name = "Bob"
```

```
age = 25
```

```
print("My name is {} and I am {} years  
old.".format(name, age))
```

Output:

My name is Bob and I am 25 years old.

Positional and Keyword Arguments:

- You can use positional arguments (based on their order) or keyword arguments (based on their names).

Python

```
print("My name is {1} and I am {0} years  
old.".format(age, name)) #positional
```

```
print("My name is {name} and I am {age} years  
old.".format(name="Charlie", age=28)) #keyword
```

Output:

My name is Charlie and I am 28 years old.

Formatting Specifiers:

- Similar to f-strings, you can use format specifiers within the curly braces.

Python

```
pi = 3.14159265
```

```
print("Pi to two decimal places: {:.2f}".format(pi))
```

Output:

Pi to two decimal places: 3.14

Choosing Between f-strings and .format():

- f-strings are generally preferred for their conciseness and readability, especially for simple formatting.

- f-strings are also generally faster.
- .format() is still useful for more complex formatting scenarios, especially when you need to reuse format strings or when you're working with older Python versions.
- If you need to define the format string outside of the print statement, then format is very useful.

Lab:

Q1. Write a Python program to print a formatted string using print() and f-string.

```
def format_and_print(name, age, city, score): """
Formats and prints a string using f-strings. Args:
name: The person's name (string). age: The
person's age (integer). city: The person's city
(string). score: The person's score (float). """
formatted_string = f"Name: {name}, Age: {age},
City: {city}, Score: {score:.2f}"
print(formatted_string) def
format_and_print_with_alignment(items, prices):
""" Formats and prints a table-like structure with
```

```

alignment. """ print(f"{'Item':<10}{'Price':>10}")
#header. <10 left aligns, >10 right aligns print("-"
* 21) #separator for item, price in zip(items,
prices): print(f"{'item':<10} ${price:>9.2f}")
#format prices to 2 decimal places # Example
usage: format_and_print("Alice", 30, "New York",
95.789) items = ["Apple", "Banana", "Cherry"]
prices = [1.25, 0.75, 2.00]
format_and_print_with_alignment(items, prices)

```

Practical Example:

Q1. Write a Python program to print “Hello, World!” on the screen.

Python

```
print("Hello, World!")
```

2. Reading Data from Keyboard

Theory:

Q1. Using the input() function to read user input from the keyboard.

```
# Get user input and store it in a variable name =
input("Enter your name: ") # Get user input for an
```

```
integer try: age = int(input("Enter your age: "))
#the int() function attempts to convert the string
to an int. except ValueError: print("Invalid input.
Please enter a number for age.") age = 0 #sets
age to 0 if the user enters a non-number. # Get
user input for a float try: height =
float(input("Enter your height in meters: "))
except ValueError: print("Invalid input. Please
enter a number for height.") height = 0.0 # Print
the collected information print(f"Name: {name}")
print(f"Age: {age}") print(f"Height: {height:.2f}
meters") #format height to 2 decimal places
#demonstrate using input for multiple inputs on
the same line. numbers = input("Enter two
numbers separated by a space: ") numbers_list =
numbers.split() #split the string at the spaces. if
len(numbers_list) == 2: try: num1 =
int(numbers_list[0]) num2 = int(numbers_list[1])
print(f"The sum of {num1} and {num2} is {num1 +
num2}") except ValueError: print("Invalid
numbers entered") else: print("Please enter
exactly two numbers, separated by a space.")
```

Q2. Converting user input into different data types (e.g., int, float, etc.).

```
def convert_user_input(): """ Demonstrates
converting user input into different data types. """
# Input as a string (default) string_input =
input("Enter a string: ") print(f"String input:
{string_input}, type: {type(string_input)}") # Input
as an integer while True: #loop until valid input
try: int_input = int(input("Enter an integer: "))
print(f"Integer input: {int_input}, type:
{type(int_input)}") break #exit loop on success
except ValueError: print("Invalid input. Please
enter an integer.") # Input as a float while True:
try: float_input = float(input("Enter a float: "))
print(f"Float input: {float_input}, type:
{type(float_input)}") break except ValueError:
print("Invalid input. Please enter a float.") # Input
as a boolean (less common, but possible) while
True: bool_input_str = input("Enter 'True' or
'False': ").lower() #convert to lowercase to
accept variations. if bool_input_str == "true":
bool_input = True print(f"Boolean input:
```

```

{bool_input}, type: {type(bool_input)})" break elif
bool_input_str == "false": bool_input = False
print(f"Boolean input: {bool_input}, type:
{type(bool_input)})" break else: print("Invalid
input. Please enter 'True' or 'False'.") # Input as a
list of integers (example of more complex
parsing) while True: try: list_input_str =
input("Enter a list of integers, comma-separated:
") list_input = [int(x.strip()) for x in
list_input_str.split(",")] #list comprehension to
convert to int. print(f"List input: {list_input}, type:
{type(list_input)})" break except ValueError:
print("Invalid input. Please enter a comma-
separated list of integers.")

```

Lab:

Q1. Write a Python program to read a name and age from the user and print a formatted output.

```

def get_and_print_user_info(): """ Reads name
and age from the user and prints a formatted
output. """ name = input("Enter your name: ")
while True: # Loop until valid age input try: age =

```

```
int(input("Enter your age: ")) if age < 0: print("Age
cannot be negative. Please enter a valid age.")
else: break #exit the loop if the age is valid.
except ValueError: print("Invalid input. Please
enter a number for age.") print(f"Name: {name},
Age: {age}") print(f"{name} is {age} years old.")
print("Information: Name - {}, Age -
{}".format(name, age)) #demonstrate format() as
well. get_and_print_user_info()
```

Practical Example:

Q1. Write a Python program to read a string, an integer, and a float from the keyboard and display them.

```
def read_and_display_data(): """ Reads a string,
an integer, and a float from the keyboard and
displays them. """ string_input = input("Enter a
string: ") while True: try: int_input =
int(input("Enter an integer: ")) break except
ValueError: print("Invalid input. Please enter an
integer.") while True: try: float_input =
float(input("Enter a float: ")) break except
```



```
ValueError: print("Invalid input. Please enter a  
float.") print("String:", string_input)  
print("Integer:", int_input) print("Float:",  
float_input) print(f"String: {string_input}, Integer:  
{int_input}, Float: {float_input}") #f-string version.  
read_and_display_data()
```

3. Opening and Closing Files

Theory:

Q1. • Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

In Python, you can open files using the `open()` function. The second argument to `open()` specifies the mode in which the file should be opened. Here's a breakdown of the common modes:

File Opening Modes:

- **'r' (Read Mode):**

- Opens the file for reading.
- The file pointer is placed at the beginning of the file.

- If the file doesn't exist, it raises a `FileNotFoundError`.
- This is the default mode if no mode is specified.

```
try: with open("my_file.txt", "r") as file: content =
file.read() print(content) except
FileNotFoundError: print("File not found.") ` ` `
```

- **'w' (Write Mode):**

- Opens the file for writing.
- If the file exists, its contents are truncated (deleted).
- If the file doesn't exist, it creates a new file.
- Be very careful using 'w', since it will overwrite existing files.

Python

```
with open("my_file.txt", "w") as file:
```

```
    file.write("This is some text written to the file.")
```

- **'a' (Append Mode):**

- Opens the file for appending.

- The file pointer is placed at the end of the file.
- If the file doesn't exist, it creates a new file.
- Data written to the file is added to the end of the existing content.

Python

```
with open("my_file.txt", "a") as file:
```

```
    file.write("\nThis is appended text.")
```

• **'r+' (Read and Write Mode):**

- Opens the file for both reading and writing.
- The file pointer is placed at the beginning of the file.
- Existing file contents are not truncated.
- Allows you to both read and modify the file.

Python

```
with open("my_file.txt", "r+") as file:
```

```
    content = file.read()
```

```
print("Original content:", content)

file.write("\nAdded this line using r+.")

file.seek(0) #resets the file pointer to the
begining of the file.

new_content = file.read()

print("Modified content:", new_content)
```

- **'w+' (Write and Read Mode):**

- Opens the file for both writing and reading.
- If the file exists, its contents are truncated.
- If the file doesn't exist, it creates a new file.
- Like 'w', it will overwrite existing files.

Python

```
with open("my_file.txt", "w+") as file:

    file.write("Writing and then reading.")

    file.seek(0) #reset the file pointer to the
begining of the file.

    content = file.read()

    print(content)
```

Important Notes:

- It's good practice to use the `with open(...)` statement. This ensures that the file is properly closed, even if errors occur.
- The `seek()` function is used to change the file pointer location.
- When using `'w'` or `'w+'`, be aware that you will delete the contents of any existing file.
- You can add `'b'` to any of these modes (e.g., `'rb'`, `'wb'`) to open the file in binary mode. This is important for non-text files.
- You can add `'t'` to any of these modes (e.g., `'rt'`, `'wt'`) to open the file in text mode. This is the default.

Sources and related content

Q2. Using the `open()` function to create and access files.

The `open()` function in Python is your primary tool for interacting with files. Here's a comprehensive guide on using it to create and access files:

1. Creating a File:

- To create a new file, you open it in write mode ('w') or append mode ('a').
- If the file doesn't exist, open() creates it.
- If it exists in write mode, the old content is deleted.

Python

Creating a new file and writing to it.

try:

```
with open("my_new_file.txt", "w") as file:
```

```
    file.write("This is the content of my new file.")
```

```
    print("File 'my_new_file.txt' created  
successfully.")
```

except Exception as e:

```
    print(f"An error occurred: {e}")
```

Appending to a file, creating it if it doesn't exist.

try:

```
with open("my_new_file.txt", "a") as file:
    file.write("\nThis line is appended to the file.")
    print("Data appended to 'my_new_file.txt'")
except Exception as e:
    print(f"An error occurred: {e}")
```

2. Accessing (Reading) a File:

- To read an existing file, you open it in read mode ('r').
- If the file doesn't exist, a FileNotFoundError is raised.

Python

Reading the content of a file.

try:

```
with open("my_new_file.txt", "r") as file:
    file_content = file.read()
    print("File content:")
    print(file_content)
```

```
except FileNotFoundError:
```

```
    print("File 'my_new_file.txt' not found.")
```

```
except Exception as e:
```

```
    print(f"An error occurred: {e}")
```

```
#Reading line by line.
```

```
try:
```

```
    with open("my_new_file.txt", "r") as file:
```

```
        for line in file:
```

```
            print(line.strip()) #strip removes newline  
characters.
```

```
except FileNotFoundError:
```

```
    print("File not found")
```

```
except Exception as e:
```

```
    print(f"An error occurred: {e}")
```

3. Key Concepts and Best Practices:

- **with open(...):**

- This is the recommended way to open files.
- It ensures that the file is automatically closed when the with block finishes, even if errors occur.
- This prevents resource leaks and data corruption.

- **File Modes:**

- 'r': Read mode (default).
- 'w': Write mode (truncates existing files).
- 'a': Append mode (adds to the end).
- 'r+': Read and write.
- 'w+': Write and read (truncates).
- 'b': Binary mode (e.g., 'rb', 'wb').
- 't': Text mode (default).

- **Error Handling:**

- Use try...except blocks to handle potential FileNotFoundError exceptions and other file-related errors.

- This makes your code more robust.

- **Reading Methods:**

- `file.read()`: Reads the entire file content as a string.
- `file.readline()`: Reads a single line.
- `file.readlines()`: Reads all lines into a list.
- Iterating over the file object directly (e.g., `for line in file:`) reads the file line by line, which is efficient for large files.

- **File Paths:**

- You can specify absolute or relative file paths.
- Use raw strings (e.g., `r"C:\my_file.txt"`) on Windows to avoid issues with backslashes.
- Use `os.path.join()` for cross-platform file path construction.

Q3. Closing files using `close()`.

In Python, it's crucial to close files after you're finished with them. While using the `with open(...)` statement is the recommended and safest way to handle file operations (as it automatically closes files), you can also explicitly close files using the `close()` method.

Here's how it works and why it's important:

Using `close()`:

1. Opening a File:

- You open a file using the `open()` function.

2. Performing Operations:

- You perform read or write operations on the file.

3. Closing the File:

- You call the `close()` method on the file object to close it.

Python

```
file = open("my_file.txt", "w") # Open for writing
file.write("This is some text.")
```

```
file.close() # Close the file
```

```
file = open("my_file.txt", "r") #open for reading.
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

Why Closing Files Is Important:

- **Resource Management:**

- Operating systems have limits on the number of files that can be open simultaneously.
- Failing to close files can lead to resource leaks and eventually cause your program to crash.

- **Data Integrity:**

- When you write to a file, the data might be buffered (held in memory) before being written to the disk.

- Closing the file ensures that all buffered data is flushed (written) to the disk, preventing data loss or corruption.

- **File Locking:**

- While a file is open, it might be locked, preventing other programs or processes from accessing it.
- Closing the file releases the lock.

- **Avoiding Errors:**

- If you attempt to open a file that is already open, you may get errors.

- **The with open(...) Alternative:**

- The with statement automatically closes the file, even if exceptions occur.
- It's generally preferred over explicitly calling close().

Python

try:

with open("my_file.txt", "r") as file:

```
    content = file.read()

    print(content)

except FileNotFoundError:

    print("File not found")

except Exception as e:

    print(f"An error occurred: {e}")
```

Key Points:

- While close() is available, using with open(...) is the recommended practice.
- Always ensure that files are closed to prevent resource leaks and data corruption.
- If you do not use the with statement, it is very important to use a try finally block to ensure that the file is closed.

Python

```
file = None
```

```
try:
```

```
    file = open("my_file.txt", "r")
```

```
    content = file.read()

    print(content)

except FileNotFoundError:

    print("File not found.")

except Exception as e:

    print(f"An error occurred: {e}")

finally:

    if file is not None:

        file.close()
```

Lab:

Q1. Write a Python program to open a file in write mode, write some text, and then close it.

```
def write_to_file(filename, text): """ Opens a file
in write mode, writes the given text, and closes it.
Args: filename: The name of the file to write to.
text: The text to write to the file. """ try: with
open(filename, "w") as file: file.write(text)
print(f"Successfully wrote to '{filename}'.")
except Exception as e: print(f"An error occurred:
```

{e}") # Example usage:

```
write_to_file("my_text_file.txt", "This is some text  
written to the file.") #Demonstrate appending  
instead of overwriting.
```

```
write_to_file("my_text_file.txt", "\nThis text is  
appended.") #\n adds a newline.
```

Practical Example:

Q1. Write a Python program to create a file and write a string into it

```
def create_and_write_file(filename, content): """  
Creates a file and writes a string into it. Args:  
filename: The name of the file to create and write  
to. content: The string to write into the file. """ try:  
with open(filename, "w") as file:  
file.write(content) print(f"File '{filename}' created  
and content written successfully.") except  
Exception as e: print(f"An error occurred: {e}") #
```

Example usage:

```
create_and_write_file("my_string_file.txt", "This  
is a string written to the file.") #Example with a  
different string.
```



```
create_and_write_file("another_string_file.txt",  
"This is another string. With newlines.\nAnd  
more text.")
```

4. Reading and Writing Files

Theory:

Q1. Reading from a file using `read()`, `readline()`, `readlines()`.

Python offers three primary methods for reading data from a file: `read()`, `readline()`, and `readlines()`. Each method serves a different purpose and returns data in a distinct format.

1. `read()`:

- Reads the entire content of the file as a single string.
- If a size argument is provided (`read(size)`), it reads only that many bytes.
- Useful for small files or when you need the whole file content as a string.

Python

try:

```
with open("my_file.txt", "r") as file:
```

```
    content = file.read()
```

```
    print("Using read():")
```

```
    print(content)
```

except FileNotFoundError:

```
    print("File not found.")
```

2. readline():

- Reads a single line from the file, including the newline character (\n).
- Subsequent calls to readline() read subsequent lines.
- Returns an empty string (") when the end of the file is reached.
- Useful for processing files line by line.

Python

try:

```
with open("my_file.txt", "r") as file:
```

```
print("\nUsing readline():")  
line1 = file.readline()  
line2 = file.readline()  
print(line1.strip()) #strip to remove the  
newline character.  
print(line2.strip())  
except FileNotFoundError:  
    print("File not found.")
```

#Read the entire file line by line using a while loop.

```
try:  
    with open("my_file.txt", "r") as file:  
        print("\nUsing readline() in a loop:")  
        line = file.readline()  
        while line:  
            print(line.strip())  
            line = file.readline()
```

except FileNotFoundError:

print("File not found.")

3. readlines():

- Reads all lines from the file and returns them as a list of strings, with each string representing a line (including the newline character).
- Useful for processing files where you need all lines in a list.

Python

try:

with open("my_file.txt", "r") as file:

lines = file.readlines()

print("\nUsing readlines():")

for line in lines:

print(line.strip())

except FileNotFoundError:

print("File not found.")

Key Considerations:

- **File Size:**

- `read()` can consume a lot of memory for very large files.
- `readline()` and `readlines()` are more memory-efficient for large files, especially when processing them line by line.

- **Newline Characters:**

- `readline()` and `readlines()` include newline characters (`\n`) at the end of each line.
- Use `.strip()` to remove these characters if needed.

- **Error Handling:**

- Always use `try...except` blocks to handle potential `FileNotFoundError` exceptions.

- **Choosing the Right Method:**

- Use `read()` for small files or when you need the entire file content as a string.
- Use `readline()` when you need to process files line by line.

- Use `readlines()` when you need all lines in a list for further processing.

- **Iteration:**

- It is also possible to iterate directly over a file object, which is the most memory efficient way to read a file line by line.

Python

try:

```
with open("my_file.txt", "r") as file:
```

```
    print("\nIterating over file object:")
```

```
    for line in file:
```

```
        print(line.strip())
```

except `FileNotFoundError`:

```
    print("File not found.")
```

Q2. Writing to a file using `write()` and `writelines()`.

Python provides two primary methods for writing data to a file: `write()` and `writelines()`.

1. `write()`:

- Writes a single string to the file.
- It does not automatically add newline characters, so you must include them explicitly (`\n`) if needed.
- Useful for writing individual strings or small amounts of data.

Python

try:

```
with open("my_output_file.txt", "w") as file:
```

```
    file.write("This is the first line.\n")
```

```
    file.write("This is the second line.")
```

```
print("Data written successfully.")
```

except Exception as e:

```
    print(f"An error occurred: {e}")
```

2. writelines():

- Writes a list of strings to the file.
- Like `write()`, it does not automatically add newline characters, so you must include them in the strings within the list.

- Useful for writing multiple lines or a collection of strings.

Python

try:

```
lines_to_write = ["Line 1\n", "Line 2\n", "Line 3"]
```

```
with open("my_output_file.txt", "w") as file:
```

```
    file.writelines(lines_to_write)
```

```
    print("Lines written successfully.")
```

except Exception as e:

```
    print(f"An error occurred: {e}")
```

#Append Example

try:

```
more_lines = ["\nAppend line 1", "\nAppend line 2"]
```

```
with open("my_output_file.txt", "a") as file:
```

```
    file.writelines(more_lines)
```



```
print("Lines appended successfully.")
```

except Exception as e:

```
print(f"An error occurred: {e}")
```

Key Considerations:

- **Newline Characters:**

- Neither `write()` nor `writelines()` adds newline characters automatically.
- You must include `\n` in your strings to create new lines in the file.

- **Data Types:**

- `write()` accepts only strings as input.
- `writelines()` accepts a list of strings.

- **File Modes:**

- Use 'w' mode to overwrite existing files or create new ones.
- Use 'a' mode to append to existing files.

- **Error Handling:**

- Always use try...except blocks to handle potential file-related errors.

- **Efficiency:**

- For writing large amounts of data, writelines() might be slightly more efficient than calling write() multiple times.
- However, for simple string writing, write() is perfectly adequate.

- **with open():**

- It is very important to use the with open() statement to ensure that the file is closed properly, even in case of errors.

Lab:

Q1. Write a Python program to read the contents of a file and print them on the console.

```
def read_and_print_file(filename): """ Reads the
contents of a file and prints them to the console.
Args: filename: The name of the file to read. """
try: with open(filename, "r") as file: content =
file.read() print(content) except
```

```

FileNotFoundError: print(f"Error: File '{filename}'
not found.") except Exception as e: print(f"An
error occurred: {e}") # Example usage:
read_and_print_file("my_file.txt") # Replace
"my_file.txt" with your file's name. #Example to
show line by line printing. def
read_and_print_file_line_by_line(filename): """
Reads the contents of a file line by line and prints
them to the console. Args: filename: The name of
the file to read. """ try: with open(filename, "r") as
file: for line in file: print(line.strip()) #strip to
remove newline characters except
FileNotFoundError: print(f"Error: File '{filename}'
not found.") except Exception as e: print(f"An
error occurred: {e}")
read_and_print_file_line_by_line("my_file.txt")

```

Q2. Write a Python program to write multiple strings into a file.

```

def write_multiple_strings_to_file(filename,
strings): """ Writes multiple strings to a file. Args:
filename: The name of the file to write to. strings:
A list of strings to write to the file. """ try: with

```

```

open(filename, "w") as file: for string in strings:
file.write(string + "\n") # Add newline character
after each string print(f"Successfully wrote
{len(strings)} strings to '{filename}':") except
Exception as e: print(f"An error occurred: {e}") #
Example usage: strings_to_write = [ "This is the
first string.", "This is the second string.", "This is
the third string.", "And this is the last one." ]
write_multiple_strings_to_file("my_strings_file.tx
t", strings_to_write) #Example using writelines.
def write_multiple_strings_writelines(filename,
strings): try: with open(filename, 'w') as file:
file.writelines([s + '\n' for s in strings]) #add
newlines in a list comprehension.
print(f"Successfully wrote {len(strings)} strings to
'{filename}' using writelines.") except Exception
as e: print(f"An error occurred: {e}")
write_multiple_strings_writelines("my_strings_fil
e2.txt", strings_to_write)

```

Practical Examples:

Q1. Write a Python program to create a file and print the string into the file.

```
def create_file_and_write_string(filename,
text_to_write): """ Creates a file and writes a
string to it. Args: filename: The name of the file to
create. text_to_write: The string to write to the
file. """ try: with open(filename, "w") as file:
file.write(text_to_write) print(f"Successfully
wrote string to '{filename}'.") except Exception as
e: print(f"An error occurred: {e}") # Example
usage:
```

```
create_file_and_write_string("my_output.txt",
"Hello, this is a string written to a file.") #
```

Example with different string.

```
create_file_and_write_string("another_file.txt",
"This is another example.\nWith multiple lines.")
```

Q2. Write a Python program to read a file and print the data on the console

```
def read_file_and_print_to_console(filename):
""" Reads the contents of a file and prints them
to the console. Args: filename: The name of the
file to read. """ try: with open(filename, "r") as
file: file_content = file.read() print(file_content)
except FileNotFoundError: print(f"Error: File
```

```
'{filename}' not found.") except Exception as e:
print(f"An error occurred: {e}") # Example usage:
read_file_and_print_to_console("my_data.txt") #
Replace "my_data.txt" with your file's name
#Example of line by line reading. def
read_file_line_by_line(filename): try: with
open(filename, 'r') as file: for line in file:
print(line.strip()) #strip to remove newlines.
except FileNotFoundError: print(f"Error: File
'{filename}' not found.") except Exception as e:
print(f"An error occurred: {e}")
read_file_line_by_line("my_data.txt")
```

Q3. Write a Python program to check the current position of the file cursor using tell().

```
def check_file_cursor_position(filename): """
Opens a file, performs some read/write
operations, and checks the file cursor position
using tell(). Args: filename: The name of the file to
work with. """ try: with open(filename, "r+") as
file: # Open for reading and writing print(f"Initial
cursor position: {file.tell()}") file.write("This is
some text.") print(f"Cursor position after writing:
```

```
{file.tell()})" file.seek(0) # Move cursor to the
beginning print(f"Cursor position after seek(0):
{file.tell()})" file.read(5) # Read 5 bytes
print(f"Cursor position after reading 5 bytes:
{file.tell()})" file.readline() print(f"Cursor position
after readline(): {file.tell()})" except
FileNotFoundError: print(f"Error: File '{filename}'
not found.") except Exception as e: print(f"An
error occurred: {e}") # Example usage (create a
dummy file first): with open("cursor_test.txt",
"w") as f: f.write("This is a test file.\nWith
multiple lines.")
check_file_cursor_position("cursor_test.txt")
```

5.Exception Handling

Theory:

Q1. Introduction to exceptions and how to handle them using try, except, and finally.

Exceptions are errors that occur during the execution of a Python program. When an exception occurs, the normal flow of the program is disrupted. Exception handling is the process of

anticipating and responding to these errors to prevent program crashes and ensure robustness. Python provides the try, except, and finally keywords for exception handling.

1. Basic Exception Handling (try...except)

- The try block contains the code that might raise an exception.
- The except block specifies how to handle a particular exception.
- If an exception occurs within the try block, Python looks for a matching except block.
- If a match is found, the code within the except block is executed.

Python

try:

```
    result = 10 / 0 # This will raise a  
ZeroDivisionError
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero is not allowed.")
```



```
except ValueError: #example of another except
block.
```

```
    print("Error: Invalid value.")
```

```
except Exception as e: # Catch any other
exception.
```

```
    print(f"An unexpected error occurred: {e}")
```

```
print("Program continues...") #The program
continues after the exception is handled.
```

2. Handling Multiple Exceptions

- You can have multiple except blocks to handle different types of exceptions.
- Python checks each except block in order, and executes the first one that matches the exception.
- You can also catch multiple exceptions in a single except block using tuples:

Python

try:

```
value = int("abc") # This will raise a ValueError
except (ValueError, TypeError):
    print("Error: Invalid input type.")
```

- A bare except: will catch all exceptions, but this is generally discouraged, as it can hide unexpected errors. It is better to catch explicit exceptions, or at the very least use except Exception as e:.

3. The finally Block

- The finally block contains code that will always be executed, regardless of whether an exception occurred or not.
- It's ¹ typically used for cleanup ² actions, such as closing files or releasing resources.

[1. stlplaces.com](http://1.stlplaces.com)

stlplaces.com

[2. codenp.com](https://2.codenp.com)

codenp.com

Python

try:

```
file = open("my_file.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

except FileNotFoundError:

```
print("File not found.")
```

finally:

```
if 'file' in locals() and file: #only close if the file  
was opened.
```

```
file.close()
```

```
print("File operation completed.")
```

4. The else Block

- The else block is executed if no exceptions occur in the try block.
- It is placed after the except blocks and before the finally block (if present).

Python

try:

```
    num = int(input("Enter a number: "))
```

except ValueError:

```
    print("Invalid input.")
```

else:

```
    print(f"You entered: {num}")
```

finally:

```
    print("Done.")
```

5. Raising Exceptions

- You can explicitly raise exceptions using the raise keyword.

- This is useful for signaling errors in your own code or re-raising exceptions after handling them.

Python

```
def divide(x, y):
```

```
    if y == 0:
```

```
        raise ZeroDivisionError("Cannot divide by  
zero.")
```

```
    return x / y
```

```
try:
```

```
    result = divide(10, 0)
```

```
except ZeroDivisionError as e:
```

```
    print(f"Error: {e}")
```

Q2. Understanding multiple exceptions and custom exceptions.

Multiple Exceptions:

Python allows you to handle multiple exceptions in several ways within a try...except block,

providing flexibility and clarity in your error-handling logic:

1. Multiple except Blocks:

- This is the most common and recommended approach.
- Each except block handles a specific exception type.
- Python checks each except block sequentially and executes the first one that matches the raised exception.

Python

try:

```
x = int(input("Enter a number: "))
```

```
result = 10 / x
```

```
print(result)
```

except ValueError:

```
    print("Error: Invalid input. Please enter a  
number.")
```

except ZeroDivisionError:

```
print("Error: Cannot divide by zero.")  
  
except Exception as e: #Always use this as a  
final catch-all.
```

```
print(f"An unexpected error occurred: {e}")
```

2. Handling Multiple Exceptions in a Single except Block:

- You can use a tuple of exception types within a single except block to handle them collectively.
- This is useful when you want to execute the same error-handling code for multiple related exceptions.

Python

try:

```
x = int(input("Enter a number: "))  
  
result = 10 / x  
  
print(result)  
  
except (ValueError, ZeroDivisionError) as e:  
  
    print(f"Error: Invalid operation. {e}")
```

except Exception as e:

```
print(f"An unexpected error occurred: {e}")
```

3. Catching All Exceptions (Generally Discouraged):

- A bare except: block catches all exceptions, but this is usually bad practice.
- It can hide unexpected errors and make debugging difficult.
- It is better to catch explicit exceptions. If you must catch everything, use except Exception as e:.

Python

try:

```
# Some code that might raise any exception
```

```
pass
```

except Exception as e:

```
print(f"An error occurred: {e}")
```

Custom Exceptions:

- Custom exceptions allow you to define your own exception types, tailored to the specific needs of your application.
- This makes your code more readable, maintainable, and expressive.
- Custom exceptions are created by defining a new class that inherits from the built-in Exception class or one of its subclasses.¹

[1. github.com](https://1.github.com)

github.com

Creating Custom Exceptions:

Python

```
class MyCustomError(Exception):
```

```
    """
```

```
    Custom exception class.
```

```
"""
```

```
def __init__(self, message):  
    super().__init__(message)  
    self.message = message
```

```
def process_data(data):  
    if not isinstance(data, list):  
        raise MyCustomError("Data must be a list.")  
  
    # Process the list.  
  
    print("Data processed successfully.")
```

```
try:  
    process_data("Not a list")  
except MyCustomError as e:  
    print(f"Custom error: {e.message}")
```

```
try:  
    process_data([1,2,3])
```

```
except MyCustomError as e:
```

```
    print(f"Custom error: {e.message}")
```

Key Points:

- **Inheritance:** Custom exceptions should inherit from Exception or a subclass.
- **Initialization:** You can add custom attributes to your exceptions in the `__init__` method.
- **Raising:** Use the raise keyword to raise custom exceptions.
- **Clarity:** Custom exceptions make your code more self-documenting.
- **Organization:** They help organize your error-handling logic.
- **Information:** They allow you to add more information to your exceptions than the standard ones allow.
- **Readability:** They improve the readability of your code.

Lab:

Q1. Write a Python program to handle exceptions in a simple calculator (division by zero, invalid input)

Python

```
def calculator():
```

```
    """
```

A simple calculator that handles division by zero and invalid input exceptions.

```
    """
```

```
    try:
```

```
        num1 = float(input("Enter the first number:"))
```

```
        num2 = float(input("Enter the second number: "))
```

```
        operation = input("Enter the operation (+, -, *, /): ")
```

```
        if operation == "+":
```

```
            result = num1 + num2
```

```
elif operation == "-":
    result = num1 - num2
elif operation == "*":
    result = num1 * num2
elif operation == "/":
    if num2 == 0:
        raise ZeroDivisionError("Cannot divide by
zero.")
    result = num1 / num2
else:
    raise ValueError("Invalid operation.")

print(f"Result: {result}")

except ValueError as ve:
    print(f"Error: {ve}")
except ZeroDivisionError as zde:
    print(f"Error: {zde}")
```

```
except Exception as e:
```

```
    print(f"An unexpected error occurred: {e}")
```

```
calculator()
```

Explanation:

1. calculator() Function:

- Prompts the user to enter two numbers and an operation.
- Uses a try...except block to handle potential exceptions.

2. Input and Operation:

- `float(input(...))` attempts to convert user input to floating-point numbers.
- The program checks the entered operation and performs the corresponding calculation.

3. Exception Handling:

- **ValueError:**

- This exception is raised if the user enters non-numeric input for the numbers or an invalid operation.
- The except ValueError block catches this exception and prints an error message.
- **ZeroDivisionError:**
 - This exception is raised if the user attempts to divide by zero.
 - The except ZeroDivisionError block catches this exception and prints an error message.
- **Exception as e:**
 - This catches any other unexpected errors, providing a general error message. this is very important.

4. Output:

- If no exceptions occur, the program prints the result of the calculation.

Example Usage:

- **Valid Input:**

- User enters: 10, 2, /
- Output: Result: 5.0

- **Division by Zero:**

- User enters: 10, 0, /
- Output: Error: Cannot divide by zero.

- **Invalid Input:**

- User enters: abc, 2, +
- Output: Error: could not convert string to float: 'abc'
- User enters: 10, 2, ^
- Output: Error: Invalid operation.

Q2. Write a Python program to demonstrate handling multiple exceptions

```
def demonstrate_multiple_exceptions(input_str,
divisor):
```

```
    """
```


Demonstrates handling multiple exceptions in a single try-except block.

Args:

input_str: A string that might be converted to an integer.

divisor: A number that might be zero.

"""

try:

num = int(input_str)

result = 10 / divisor

print(f"Result: {result}")

except ValueError:

print("Error: Invalid input string. Please enter an integer.")

except ZeroDivisionError:

```
print("Error: Division by zero is not allowed.")
```

```
except TypeError:
```

```
    print("Error: Type Error. Check the type of the  
divisor.")
```

```
except Exception as e: # Catch any other  
errors.
```

```
    print(f"An unexpected error occurred: {e}")
```

```
# Example usage:
```

```
demonstrate_multiple_exceptions("abc", 2) #  
Triggers ValueError
```

```
demonstrate_multiple_exceptions("10", 0) #  
Triggers ZeroDivisionError
```

```
demonstrate_multiple_exceptions("10", "a") #  
Triggers TypeError.
```

```
demonstrate_multiple_exceptions("10", 2) # No  
exception
```

`demonstrate_multiple_exceptions(None, 2) #`
Triggers `TypeError`.

Explanation:

1. `demonstrate_multiple_exceptions(input_str, divisor)` Function:

- Takes an `input_str` (which might not be a valid integer) and a `divisor` (which might be zero or a non-number) as input.
- Uses a `try...except` block to handle multiple exception types.

2. `try` Block:

- `num = int(input_str)`: Attempts to convert the input string to an integer.
- `result = 10 / divisor`: Attempts to divide 10 by the given divisor.

3. `except` Blocks:

- **`ValueError`**: Catches exceptions that occur when the input string cannot be converted to an integer.

- **ZeroDivisionError:** Catches exceptions that occur when the divisor is zero.
- **TypeError:** Catches exceptions that occur when the divisor is not the correct type.
- **Exception as e:** Catches any other exceptions that may occur. This is very important, even when you handle many other exceptions, as unexpected exceptions can occur.

4. Example Usages:

- The example calls demonstrate how the different except blocks are triggered based on the inputs.
- This makes it easy to see how multiple exceptions are handled.
- The example with None demonstrates how unexpected type errors are caught.

Practical Examples:

Q1. Write a Python program to handle exceptions in a calculator

```
def calculator():
```

```
    """
```

A calculator program that handles potential exceptions.

```
    """
```

```
    try:
```

```
        num1 = float(input("Enter the first number:"))
```

```
        num2 = float(input("Enter the second number: "))
```

```
        operation = input("Enter the operation (+, -, *, /): ")
```

```
        if operation == "+":
```

```
            result = num1 + num2
```

```
        elif operation == "-":
```

```
            result = num1 - num2
```

```
        elif operation == "*":
```

```
        result = num1 * num2

    elif operation == "/":

        if num2 == 0:

            raise ZeroDivisionError("Division by zero
is not allowed.")

        result = num1 / num2

    else:

        raise ValueError("Invalid operation
entered.")

    print(f"Result: {result}")

except ValueError as ve:

    print(f"Error: {ve}") # Handles invalid number
or operation input

except ZeroDivisionError as zde:

    print(f"Error: {zde}") # Handles division by
zero
```

```
except Exception as e:  
    print(f"An unexpected error occurred: {e}") #  
Handles any other errors.
```

```
# Run the calculator  
calculator()
```

```
# example of another way to handle the  
exceptions.
```

```
def calculator_alternative():  
    try:  
        num1 = float(input("Enter first number: "))  
        num2 = float(input("Enter second number: "))  
        operation = input("Enter operation (+, -, *, /): ")  
  
        if operation == '/':  
            result = num1 / num2  
  
        elif operation == '*':
```

```
    result = num1 * num2
elif operation == '+':
    result = num1 + num2
elif operation == '-':
    result = num1 - num2
else:
    raise ValueError("Invalid operator")

print(f"Result: {result}")

except ValueError as e:
    print(f"Error: {e}")
except ZeroDivisionError:
    print("Error: cannot divide by zero")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

calculator_alternative()
```


Explanation:

1. calculator() Function:

- Takes user input for two numbers and an operation.
- Uses a try...except block to handle exceptions.
- **Input Conversion:**
 - float(input(...)) attempts to convert the input to floating-point numbers.
 - If the input is not a valid number, a ValueError is raised.
- **Operation Handling:**
 - The code checks the operation and performs the corresponding calculation.
 - If the operation is / and the second number is 0, a ZeroDivisionError is raised.

- If the operation is invalid, a `ValueError` is raised.

- **Exception Handling:**

- `except ValueError as ve:` handles invalid number or operation input.
- `except ZeroDivisionError as zde:` handles division by zero.
- `except Exception as e:` handles any other unexpected errors. This is very important.

- **Output:**

- If no exceptions occur, the result is printed.

2.calculator_alternative() Function:

- This function does the same thing as the first function, but it shows an alternative way to organize the code.
- It also demonstrates how to catch a bare `ZeroDivisionError`, instead of `ZeroDivisionError as zde`.

- Both ways are valid.

3. Example Usage:

- **Valid Input:**
 - Input: 10, 5, /
 - Output: Result: 2.0
- **Division by Zero:**
 - Input: 10, 0, /
 - Output: Error: Division by zero is not allowed.
- **Invalid Input (Number):**
 - Input: abc, 5, +
 - Output: Error: could not convert string to float: 'abc'
- **Invalid Input (Operation):**
 - Input: 10, 5, ^
 - Output: Error: Invalid operation entered.

Q2, Write a Python program to handle multiple exceptions (e.g., file not found, division by zero).

```
def handle_multiple_exceptions(filename,  
numerator, denominator):
```

```
    """
```

Handles multiple exceptions:
FileNotFoundError and ZeroDivisionError.

Args:

filename: The name of the file to read from.

numerator: The numerator for division.

denominator: The denominator for division.

```
    """
```

```
try:
```

```
    with open(filename, "r") as file:
```

```
        content = file.read()
```

```
        print(f"File content: {content}")
```

```
result = numerator / denominator
```

```
print(f"Division result: {result}")
```

```
except FileNotFoundError:
```

```
    print(f"Error: File '{filename}' not found.")
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero is not allowed.")
```

```
except TypeError:
```

```
    print("Error: Type error, please check your  
input types.")
```

```
except Exception as e: #Always catch any  
other exceptions.
```

```
    print(f"An unexpected error occurred: {e}")
```

```
# Example usage:
```

```
# Create a dummy file for testing
```

```
with open("test_file.txt", "w") as f:
```

```
    f.write("This is a test file.")
```

```
handle_multiple_exceptions("test_file.txt", 10, 2)
```

```
# No exception
```

```
handle_multiple_exceptions("nonexistent_file.txt", 10, 2) # FileNotFoundError
```

```
handle_multiple_exceptions("test_file.txt", 10, 0) # ZeroDivisionError
```

```
handle_multiple_exceptions("test_file.txt", "abc", 2) #TypeError
```

```
handle_multiple_exceptions("test_file.txt", None, 2) #TypeError
```

Explanation:

1.handle_multiple_exceptions(filename, numerator, denominator) Function:

- Takes the filename, numerator, and denominator as input.
- Uses a try...except block to handle potential exceptions.

2.try Block:

- `with open(filename, "r") as file::` Attempts to open the specified file for reading.
- `content = file.read():` Reads the content of the file.
- `result = numerator / denominator:` Attempts to perform the division.

3. **except Blocks:**

- **FileNotFoundError:** Catches exceptions that occur when the specified file does not exist.
- **ZeroDivisionError:** Catches exceptions that occur when the denominator is zero.
- **TypeError:** Catches exceptions that occur when the input types are incorrect.
- **Exception as e:** Catches any other exceptions that might occur. This is very important.

4. **Example Usage:**

- The example calls demonstrate how the different except blocks are triggered based on the inputs.
- The creation of test_file.txt allows for testing of the FileNotFoundError.
- The TypeError examples show how incorrect types are caught.

5. Important Notes:

- It is generally good practice to handle specific exceptions whenever possible, as this makes your code more robust and easier to debug.
- Always include a catch-all except Exception as e: at the end of your exception handling blocks.
- The with open() statement ensures that the file is closed properly.

Q3. Write a Python program to handle file exceptions and use the finally block for closing the file.


```
def process_file(filename):
```

```
    """
```

Handles file exceptions and uses a finally block to close the file.

Args:

filename: The name of the file to process.

```
    """
```

```
    file = None # Initialize file variable outside the  
    try block
```

```
    try:
```

```
        file = open(filename, "r")
```

```
        content = file.read()
```

```
        print(f"File content:\n{content}")
```

```
    except FileNotFoundError:
```

```
        print(f"Error: File '{filename}' not found.")
```

```
    except Exception as e:
```

```
        print(f"An unexpected error occurred: {e}")
    finally:
        if file: # Check if the file was opened before
closing
            file.close()
            print("File closed in the finally block.")
        else:
            print("No file opened to close.")
```

Example usage:

Create a dummy file for testing

with open("test_file.txt", "w") as f:

```
    f.write("This is a test file.")
```

```
process_file("test_file.txt") # File exists
```

```
process_file("nonexistent_file.txt") # File does
not exist
```

`process_file(123)` #Will raise a `TypeError`,
showing the `finally` block still executes.

Explanation:

1. `process_file(filename)` Function:

- Takes the filename as input.
- Initializes the file variable to `None` outside the try block. This is crucial because the `finally` block needs to access it.
- **try Block:**
 - `file = open(filename, "r")`: Attempts to open the file in read mode.
 - `content = file.read()`: Reads the file content.
 - `print(f"File content:\n{content}")`: Prints the content.
- **except Blocks:**
 - `FileNotFoundError`: Handles the case where the file is not found.

- Exception as e: Handles any other exceptions that might occur.
- **finally Block:**
 - if file:: Checks if the file was successfully opened. This prevents errors if open() failed.
 - file.close(): Closes the file.
 - print("File closed in the finally block."): Confirms that the file was closed.
 - else: If the file was not opened, then print a message saying that.

2. Example Usage:

- A dummy file "test_file.txt" is created for testing.
- process_file("test_file.txt"): Demonstrates normal file processing.
- process_file("nonexistent_file.txt"): Demonstrates FileNotFoundError handling.

- `process_file(123)`: Demonstrates that the `finally` block will still run even if a type error is thrown.

3. `finally` Block Importance:

- The `finally` block ensures that the file is closed, regardless of whether an exception occurred.
- This is essential for releasing system resources and preventing data corruption.
- The `if file: check` is vital. It prevents errors that would occur if you tried to close a file that was never successfully opened.

Q4. Write a Python program to print custom exceptions.

```
class CustomError(Exception): """ A custom
exception class. """
    def __init__(self, message,
error_code=None):
        super().__init__(message)
        self.message = message
        self.error_code =
error_code
    def __str__(self):
        if self.error_code:
            return f"CustomError (Code: {self.error_code}):
{self.message}"
        else:
            return f"CustomError:
```

```
{self.message}" def process_data(data): """ A
function that might raise a custom exception. """
if data is None: raise CustomError("Data cannot
be None.", error_code=1001) if not
isinstance(data, list): raise CustomError("Data
must be a list.") if len(data) == 0: raise
CustomError("Data list cannot be empty.",
error_code=1002) print("Data processed
successfully.") # Example usage: try:
process_data(None) except CustomError as e:
print(e) # Prints the custom exception message
try: process_data("Not a list") except
CustomError as e: print(e) try: process_data([])
except CustomError as e: print(e) try:
process_data([1, 2, 3]) # No exception except
CustomError as e: print(e)
```

6. Class and Object (OOP Concepts)

Theory:

Q1. Understanding the concepts of classes, objects, attributes, and methods in Python.

In Python, object-oriented programming (OOP) revolves around the concepts of classes and objects. Here's a breakdown of these key concepts:

1. Classes:

- A class is like a blueprint or a template for creating objects. It defines the structure and behavior that objects of that class will have.¹

[1. github.com](https://1.github.com)

github.com

- It specifies the attributes (data) and methods (functions) that the objects will possess.
- Classes are defined using the class keyword.

Python

```
class Dog:
```

```
# Class attributes and methods go here  
pass # Placeholder, we'll add more later
```

2. Objects:

- An object is an instance of a class. It's a concrete realization of the class's blueprint.
- Objects are created from classes.
- Each object has its own unique set of attributes and can use the methods defined in its class.

Python

```
my_dog = Dog() # Creating an object (instance)  
of the Dog class
```

```
another_dog = Dog() # creating another object.
```

3. Attributes:

- Attributes are variables that store data associated with an object.
- They represent the object's state or characteristics.

- Attributes can be defined within the class or dynamically added to objects.

Python

```
class Dog:
```

```
    def __init__(self, name, breed):  
        self.name = name # Instance attributes  
        self.breed = breed
```

```
my_dog = Dog("Buddy", "Golden Retriever")  
print(my_dog.name) # Accessing an attribute  
print(my_dog.breed)
```

- **__init__ method:** This special method (constructor) is called when an object is created. It's used to initialize the object's attributes.
- **self:** This parameter refers to the current object instance. It's used to access the object's attributes and methods within the class.

4. Methods:

- Methods are functions that are defined within a class.
- They define the object's behavior or actions.
- Methods can access and modify the object's attributes.

Python

```
class Dog:
```

```
    def __init__(self, name, breed):
```

```
        self.name = name
```

```
        self.breed = breed
```

```
    def bark(self):
```

```
        print("Woof!")
```

```
    def display_info(self):
```

```
        print(f"Name: {self.name}, Breed:  
{self.breed}")
```

```
my_dog = Dog("Buddy", "Golden Retriever")
```

```
my_dog.bark() # Calling a method
```

```
my_dog.display_info()
```

Key Concepts in Summary:

- **Class:** A blueprint.
- **Object:** An instance of a class.
- **Attribute:** A variable that stores data (state).
- **Method:** A function that defines behavior (actions).
- **__init__:** The constructor method for initializing objects.
- **self:** Refers to the current object instance.

Example with more details:

Python

```
class Car:
```

```
    def __init__(self, make, model, year):
```

```
        self.make = make
```

```
self.model = model  
  
self.year = year  
  
self.odometer_reading = 0 #default attribute.
```

```
def get_descriptive_name(self):  
    long_name = f"{self.year} {self.make}  
{self.model}"  
    return long_name.title()
```

```
def read_odometer(self):  
    print(f"This car has {self.odometer_reading}  
miles on it.")
```

```
def update_odometer(self, mileage):  
    if mileage >= self.odometer_reading:  
        self.odometer_reading = mileage  
    else:  
        print("You can't roll back an odometer!")
```

```
my_car = Car("Toyota", "Camry", 2022)
print(my_car.get_descriptive_name())
my_car.read_odometer()
my_car.update_odometer(100)
my_car.read_odometer()
my_car.update_odometer(50) #will print the
error message.
```

Q2. Difference between local and global variables.

In Python, the distinction between local and global variables hinges on their scope, or where they can be accessed within a program. Here's a breakdown:

Local Variables:

- **Definition:**

- Local variables are declared inside a function or a specific block of code.

- **Scope:**

- They are accessible only within the function or block where they are defined.

- **Lifetime:**

- They exist only during the execution of the function or block. Once the function or block finishes, the local variables are destroyed.

- **Example:**

Python

```
def my_function():  
    local_var = 10  
    print(local_var)
```

`my_function()` # Output: 10

`# print(local_var)` #This would cause an error.

Global Variables:

- **Definition:**

- Global variables are declared outside of any function or block of code, typically at the top of a Python script.

- **Scope:**

- They are accessible from anywhere within the program, including inside functions.

- **Lifetime:**

- They exist throughout the entire execution of the program.

- **Example:**

Python

```
global_var = 20
```

```
def my_function():
```

```
    print(global_var)
```

```
my_function() # Output: 20
```

```
print(global_var) #Output: 20
```

Key Differences Summarized:

- **Scope:**

- Local: Function or block-specific.
- Global: Program-wide.

- **Accessibility:**

- Local: Only within its defining function/block.
- Global: Anywhere in the program.

- **Lifetime:**

- Local: Function/block execution.
- Global: Program execution.

Lab:

Q1. Write a Python program to create a class and access its properties using an object

```
class Dog: """A simple class representing a
dog."""
    def __init__(self, name, breed, age):
        """Initialize name, breed, and age attributes."""
        self.name = name
        self.breed = breed
        self.age = age
    def bark(self): """Simulate a dog barking."""
```



```

print("Woof! Woof!")
def display_info(self):
    """Display information about the dog."""
    print(f"Name: {self.name}")
    print(f"Breed: {self.breed}")
    print(f"Age: {self.age}")
# Create an object (instance) of the Dog class
my_dog = Dog("Buddy", "Golden Retriever", 3)
# Access the object's properties (attributes)
print(f"My dog's name is {my_dog.name}.")
print(f"My dog is a {my_dog.breed}.")
print(f"My dog is {my_dog.age} years old.")
# Call the object's methods
my_dog.bark()
my_dog.display_info()
# Create another dog object.
another_dog = Dog("Fido", "Labrador", 5)
another_dog.display_info()
another_dog.bark()
# Modify an attribute of an object.
my_dog.age = 4
my_dog.display_info()

```

Practical Examples:

Q1.) Write a Python program to create a class and access the properties of the class using an object

```

class Book: """A class representing a book."""
    def __init__(self, title, author, publication_year):

```

```
"""Initialize the book's title, author, and
publication year.""" self.title = title self.author =
author self.publication_year = publication_year
self.is_available = True #Default attribute, all new
books are available. def display_book_info(self):
"""Display the book's information.""" print(f"Title:
{self.title}") print(f"Author: {self.author}")
print(f"Publication Year: {self.publication_year}")
print(f"Availability: {'Available' if self.is_available
else 'Checked out'}") def check_out(self):
"""Simulate checking out the book.""" if
self.is_available: self.is_available = False
print(f"'{self.title}' has been checked out.") else:
print(f"'{self.title}' is already checked out.") def
return_book(self): """Simulate returning the
book.""" if not self.is_available: self.is_available =
True print(f"'{self.title}' has been returned.") else:
print(f"'{self.title}' is already available.") # Create
an object (instance) of the Book class my_book =
Book("The Hitchhiker's Guide to the Galaxy",
"Douglas Adams", 1979) # Access the object's
properties (attributes) print(f"Book title:
```

```

{my_book.title}") print(f"Book author:
{my_book.author}") print(f"Publication year:
{my_book.publication_year}") # Call the object's
methods my_book.display_book_info()
my_book.check_out()
my_book.display_book_info() #show the
availability has changed. my_book.check_out()
#show the message for an already checked out
book. my_book.return_book()
my_book.display_book_info() #show the return.
my_book.return_book() #show the message for
an already returned book. #Create another book
object. another_book = Book("Pride and
Prejudice", "Jane Austen", 1813)
another_book.display_book_info()

```

Q2. Write a Python program to demonstrate the use of local and global variables in a class.

```

class VariableDemo: """A class to demonstrate
local and global variables.""" global_var = 10 #
Global variable (class-level) def __init__(self,
instance_var): """Initializes instance variables."""
self.instance_var = instance_var # Instance

```

```

variable def display_variables(self, local_var):
    """Displays local, global, and instance
    variables.""" print(f"Local variable: {local_var}")
    print(f"Instance variable: {self.instance_var}")
    print(f"Global variable:
    {VariableDemo.global_var}") def
    modify_variables(self, new_local, new_instance,
    new_global): """Modifies local, instance, and
    global variables.""" local_var = new_local # Local
    variable reassigned self.instance_var =
    new_instance # Instance variable modified
    VariableDemo.global_var = new_global # Global
    variable modified print("Variables after
    modification:") print(f"Modified Local variable
    (within method): {local_var}") print(f"Modified
    Instance variable: {self.instance_var}")
    print(f"Modified Global variable:
    {VariableDemo.global_var}") # Create an object
    of the VariableDemo class demo_obj =
    VariableDemo(20) # Display variables
    demo_obj.display_variables(30) # Modify the
    variables demo_obj.modify_variables(40, 50, 60)

```

```
# Display the variables again to see the changes
demo_obj.display_variables(70) # access global
variable outside of the class. print(f"Global
Variable outside of class:
{VariableDemo.global_var}")
```

7. Inheritance

Theory:

Q1. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (subclass/child class) to inherit properties and methods from another class (superclass/parent class). Python supports several types of inheritance:

1. Single Inheritance:

- A subclass inherits from a single superclass.
- This is the simplest form of inheritance.

Python

```
class Parent:
```

```
def parent_method(self):  
    print("Parent method")
```

```
class Child(Parent):  
    def child_method(self):  
        print("Child method")
```

```
child_obj = Child()  
child_obj.parent_method() # Inherited from  
Parent  
child_obj.child_method()
```

2. Multilevel Inheritance:

- A subclass inherits from a superclass, which in turn inherits from another superclass.
- This creates¹ a chain of inheritance.

[1. www.sanfoundry.com](http://www.sanfoundry.com)

www.sanfoundry.com

Python

```
class Grandparent:
    def grandparent_method(self):
        print("Grandparent method")

class Parent(Grandparent):
    def parent_method(self):
        print("Parent method")

class Child(Parent):
    def child_method(self):
        print("Child method")

child_obj = Child()
```

child_obj.grandparent_method() # Inherited
from Grandparent

child_obj.parent_method() # Inherited from
Parent

child_obj.child_method()

3. Multiple Inheritance:

- A subclass inherits from multiple superclasses.
- Python supports multiple inheritance, but it can lead to complexities (e.g., the diamond problem).

Python

```
class Parent1:
```

```
    def parent1_method(self):  
        print("Parent1 method")
```

```
class Parent2:
```

```
    def parent2_method(self):  
        print("Parent2 method")
```



```
class Child(Parent1, Parent2):  
    def child_method(self):  
        print("Child method")
```

```
child_obj = Child()  
child_obj.parent1_method()  
child_obj.parent2_method()  
child_obj.child_method()
```

4. Hierarchical Inheritance:

- Multiple subclasses inherit from a single superclass.
- This creates a hierarchy of classes.

Python

```
class Parent:  
    def parent_method(self):  
        print("Parent method")
```

```
class Child1(Parent):  
    def child1_method(self):  
        print("Child1 method")
```

```
class Child2(Parent):  
    def child2_method(self):  
        print("Child2 method")
```

```
child1_obj = Child1()  
child2_obj = Child2()
```

```
child1_obj.parent_method()  
child1_obj.child1_method()
```

```
child2_obj.parent_method()  
child2_obj.child2_method()
```

5. Hybrid Inheritance:

- A combination of two or more types of inheritance.
- For example, a class might inherit from multiple classes, and one of those classes might inherit from another class.

Python

```
class Grandparent:
```

```
    def grandparent_method(self):  
        print("Grandparent method")
```

```
class Parent1(Grandparent):
```

```
    def parent1_method(self):  
        print("Parent1 method")
```

```
class Parent2:
```

```
    def parent2_method(self):  
        print("Parent2 method")
```

```
class Child(Parent1, Parent2):
```

```
    def child_method(self):
```

```
        print("Child method")
```

```
child_obj = Child()
```

```
child_obj.grandparent_method()
```

```
child_obj.parent1_method()
```

```
child_obj.parent2_method()
```

```
child_obj.child_method()
```

Important Notes:

- **Method Resolution Order (MRO):** In multiple inheritance, Python uses the C3 linearization algorithm to determine the order in which methods are inherited. You can view the MRO of a class using `ClassName.__mro__`.
- **super():** The `super()` function is used to call methods from superclasses. It's particularly important in multiple inheritance to avoid ambiguities.

- **Diamond Problem:** Multiple inheritance can lead to the diamond problem, where a class inherits from two classes that inherit from a common superclass. Python's MRO helps to resolve this, but it's important to be aware of the potential issues.
- Careful planning is needed when using multiple inheritance. It can make code complex and hard to maintain if not used correctly.

Q2. Using the `super()` function to access properties of the parent class.

The `super()` function in Python is a powerful tool for accessing and calling methods and attributes from a parent (superclass) class within a child (subclass) class. It's particularly useful in inheritance scenarios, especially with multiple inheritance, to avoid ambiguity and promote cleaner, more maintainable code.

Here's how `super()` works and why it's essential:

Basic Usage:

- `super()` returns a temporary object of the superclass, allowing you to call its methods.
- It's commonly used within the child class's methods, especially the `__init__` constructor.

Python

```
class Parent:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        print("Parent __init__ called")
```

```
    def parent_method(self):
```

```
        print("Parent method called")
```

```
class Child(Parent):
```

```
    def __init__(self, name, age):
```

```
        super().__init__(name) # Call Parent's
```

```
    __init__
```

```
        self.age = age
```

```
print("Child __init__ called")
```

```
def child_method(self):  
    super().parent_method() # Call Parent's  
method  
    print("Child method called")
```

```
child_obj = Child("Alice", 30)  
child_obj.child_method()
```

Explanation:

1. `super().__init__(name):`

- This line calls the `__init__` method of the Parent class.
- It ensures that the parent's initialization logic is executed before the child's.
- This is crucial for proper object setup.

2. `super().parent_method():`

- This line calls the `parent_method` of the Parent class.
- It allows the child class to reuse the parent's functionality.

Benefits of `super()`:

• Avoiding Redundancy:

- It prevents you from repeating code that already exists in the parent class.

• Method Resolution Order (MRO):

- In multiple inheritance, `super()` helps to ensure that methods are called in the correct order, according to the MRO.

• Maintainability:

- It makes your code more maintainable and easier to understand, especially in complex inheritance hierarchies.

• Flexibility:

- It allows you to easily extend or modify the behavior of parent class methods.

Multiple Inheritance Example:

Python

```
class Parent1:
```

```
    def method(self):
```

```
        print("Parent1 method")
```

```
class Parent2:
```

```
    def method(self):
```

```
        print("Parent2 method")
```

```
class Child(Parent1, Parent2):
```

```
    def method(self):
```

```
        super().method() # Calls Parent1's method  
(due to MRO)
```

```
        print("Child method")
```

```
child_obj = Child()
```

```
child_obj.method()
```

```
print(Child.__mro__) #print method resolution order.
```

In this example, `super().method()` calls `Parent1`'s method because `Parent1` comes before `Parent2` in the MRO. The MRO is the order in which Python looks for methods in a class hierarchy.

Key Points:

- `super()` is essential for proper inheritance, especially in multiple inheritance.
- It avoids redundant code and ensures that parent class methods are called correctly.
- It improves code maintainability and readability.
- The method resolution order is important when using `super` with multiple inheritance.

Lab:

Q1. Write Python programs to demonstrate different types of inheritance (single, multiple, multilevel, etc.).

```

# 1. Single Inheritance class Animal: def
speak(self): print("Animal speaks") class
Dog(Animal): def bark(self): print("Woof!")
print("\n--- Single Inheritance ---") dog = Dog()
dog.speak() # Inherited from Animal dog.bark() #
2. Multiple Inheritance class Swimmer: def
swim(self): print("Can swim") class Walker: def
walk(self): print("Can walk") class
Frog(Swimmer, Walker): def jump(self):
print("Can jump") print("\n--- Multiple
Inheritance ---") frog = Frog() frog.swim()
frog.walk() frog.jump() # 3. Multilevel Inheritance
class Grandparent: def
grandparent_method(self): print("Grandparent
method") class Parent(Grandparent): def
parent_method(self): print("Parent method")
class Child(Parent): def child_method(self):
print("Child method") print("\n--- Multilevel
Inheritance ---") child = Child()
child.grandparent_method()
child.parent_method() child.child_method() # 4.
Hierarchical Inheritance class Vehicle: def

```

```

vehicle_type(self): print("Vehicle")
class Car(Vehicle):
    def car_type(self):
        print("Car")
class Bike(Vehicle):
    def bike_type(self):
        print("Bike")
print("\n--- Hierarchical Inheritance ---")
car = Car()
bike = Bike()
car.vehicle_type()
car.car_type()
bike.vehicle_type()
bike.bike_type()

# 5. Hybrid Inheritance (Combination of Multiple and Multilevel)
class Engine:
    def engine_type(self):
        print("Engine")
class ElectricEngine(Engine):
    def electric_engine_type(self):
        print("Electric Engine")
class Body:
    def body_type(self):
        print("Body")
class HybridCar(ElectricEngine, Body):
    def hybrid_car_type(self):
        print("Hybrid Car")
print("\n--- Hybrid Inheritance ---")
hybrid_car = HybridCar()
hybrid_car.engine_type()
hybrid_car.electric_engine_type()
hybrid_car.body_type()
hybrid_car.hybrid_car_type()

# 6. Demonstrating super() with multilevel inheritance.
class Grandparent2:
    def __init__(self,

```

```

grandparent_name): self.grandparent_name =
grandparent_name print(f"Grandparent
{grandparent_name} initialized") class
Parent2(Grandparent2): def __init__(self,
grandparent_name, parent_name):
super().__init__(grandparent_name)
self.parent_name = parent_name print(f"Parent
{parent_name} initialized") class
Child2(Parent2): def __init__(self,
grandparent_name, parent_name, child_name):
super().__init__(grandparent_name,
parent_name) self.child_name = child_name
print(f"Child {child_name} initialized") print("\n--
- Super() with Multilevel Inheritance ---") child2 =
Child2("Grandpa", "Dad", "Son")

```

Practical Examples:

Q1. Write a Python program to show single inheritance.

```

class Animal: """Base class representing an
animal.""" def __init__(self, name): """Initializes
the animal's name.""" self.name = name def

```

```
speak(self): """Generic speak method (to be
overridden by subclasses).""" print("Animal
sound") def display_info(self): """Displays basic
animal information.""" print(f"Name:
{self.name}") class Dog(Animal): """Derived
class representing a dog, inheriting from
Animal.""" def __init__(self, name, breed):
"""Initializes the dog's name and breed."""
super().__init__(name) # Call the parent class's
__init__ self.breed = breed def speak(self):
"""Overrides the Animal's speak method for
dogs.""" print("Woof!") def display_info(self):
"""Extends the Animal's display_info method for
dogs.""" super().display_info() # Call the parent
class's display_info print(f"Breed: {self.breed}") #
Create an object of the Dog class my_dog =
Dog("Buddy", "Golden Retriever") # Access
methods from the Dog class (including inherited
ones) my_dog.speak() my_dog.display_info()
#Create an object of the Animal class, to show it
still works. generic_animal = Animal("Generic
```

```
Animal") generic_animal.speak()  
generic_animal.display_info()
```

Q2. Write a Python program to show multilevel inheritance.

```
class Grandparent: """Base class representing a  
grandparent."""  
    def grandparent_method(self):  
        """Method specific to grandparents."""  
        print("Grandparent method called.")  
class Parent(Grandparent): """Intermediate class  
representing a parent, inheriting from  
Grandparent."""  
    def parent_method(self):  
        """Method specific to parents."""  
        print("Parent method called.")  
class Child(Parent): """Derived  
class representing a child, inheriting from  
Parent."""  
    def child_method(self):  
        """Method specific to children."""  
        print("Child method called.")  
# Create an object of the Child class  
child_obj = Child()  
# Access methods from all  
# ancestor classes  
child_obj.grandparent_method()  
child_obj.parent_method()  
child_obj.child_method() #Demonstrating
```

```

super() in a multilevel inheritance scenario. class
Grandparent2: def __init__(self,
grandparent_name): self.grandparent_name =
grandparent_name print(f"Grandparent
{grandparent_name} initialized.") class
Parent2(Grandparent2): def __init__(self,
grandparent_name, parent_name):
super().__init__(grandparent_name) # Call
Grandparent's init. self.parent_name =
parent_name print(f"Parent {parent_name}
initialized.") class Child2(Parent2): def
__init__(self, grandparent_name, parent_name,
child_name):
super().__init__(grandparent_name,
parent_name) # Call Parent's init.
self.child_name = child_name print(f"Child
{child_name} initialized.") child_object2 =
Child2("Grandpa", "Dad", "Son")

```

Q3. Write a Python program to show multiple inheritance

```

class Swimmer: """Class representing an entity
that can swim.""" def swim(self): """Simulates

```



```
swimming.""" print("Swimming...") class Walker:
"""Class representing an entity that can walk."""
def walk(self): """Simulates walking."""
print("Walking...") class Amphibian(Swimmer,
Walker): """Class representing an amphibian,
inheriting from both Swimmer and Walker.""" def
__init__(self, name): """Initializes the
amphibian's name.""" self.name = name def
display_info(self): """Displays information about
the amphibian.""" print(f"{self.name} is an
amphibian.") def jump(self): """Simulates
jumping.""" print("Jumping...") # Create an object
of the Amphibian class frog =
Amphibian("Froggy") # Access methods from
both parent classes and the Amphibian class
frog.swim() frog.walk() frog.jump()
frog.display_info() #Demonstrate super() and
method resolution order. class Base1: def
method(self): print("Base1 method") class
Base2: def method(self): print("Base2 method")
class Derived(Base1, Base2): def method(self):
super().method() #call the method of the first
```

```
parent in the MRO. print("Derived method")
derived = Derived() derived.method()
print(Derived.__mro__) #print the method
resolution order.
```

Q4. Write a Python program to show hierarchical inheritance.

```
class Vehicle: """Base class representing a
vehicle.""" def __init__(self, make, model):
"""Initializes the vehicle's make and model."""
self.make = make self.model = model def
display_info(self): """Displays basic vehicle
information.""" print(f"Make: {self.make}, Model:
{self.model}") def start_engine(self): """Generic
engine start method.""" print("Engine started.")
class Car(Vehicle): """Derived class representing
a car, inheriting from Vehicle.""" def __init__(self,
make, model, num_doors): """Initializes the car's
make, model, and number of doors."""
super().__init__(make, model) self.num_doors =
num_doors def display_info(self): """Extends the
Vehicle's display_info method for cars."""
super().display_info() print(f"Number of doors:
```

```

{self.num_doors}") def honk(self): """Simulates a
car honking.""" print("Honk! Honk!") class
Motorcycle(Vehicle): """Derived class
representing a motorcycle, inheriting from
Vehicle.""" def __init__(self, make, model,
has_sidecar): """Initializes the motorcycle's
make, model, and sidecar status."""
super().__init__(make, model) self.has_sidecar =
has_sidecar def display_info(self): """Extends
the Vehicle's display_info method for
motorcycles.""" super().display_info() print(f"Has
sidecar: {self.has_sidecar}") def wheelie(self):
"""Simulates a motorcycle doing a wheelie."""
print("Doing a wheelie!") # Create objects of the
Car and Motorcycle classes my_car =
Car("Toyota", "Camry", 4) my_motorcycle =
Motorcycle("Harley-Davidson", "Sportster",
False) # Access methods from the respective
classes (including inherited ones) print("Car:")
my_car.display_info() my_car.start_engine()
my_car.honk() print("\nMotorcycle:")
my_motorcycle.display_info()

```

```
my_motorcycle.start_engine()
my_motorcycle.wheelie() #Demonstrate that
vehicle methods still work on vehicle objects.
generic_vehicle = Vehicle("Generic", "Vehicle")
print("\nGeneric Vehicle:")
generic_vehicle.display_info()
generic_vehicle.start_engine()
```

Q5. Write a Python program to show hybrid inheritance.

```
class Vehicle: """Base class for vehicles.""" def
vehicle_info(self): print("Vehicle type.") class
Engine: """Class representing an engine.""" def
engine_type(self): print("Engine information.")
class ElectricEngine(Engine): """Class
representing an electric engine, inheriting from
Engine.""" def electric_engine_info(self):
print("Electric engine information.") class
Car(Vehicle, ElectricEngine): """Class
representing a car, inheriting from Vehicle and
ElectricEngine.""" def car_info(self): print("Car
information.") class SportsCar(Car): """Class
representing a sports car, inheriting from Car."""
```

```

def sports_car_info(self): print("Sports car
information.") # Create objects vehicle =
Vehicle() engine = Engine() electric_engine =
ElectricEngine() car = Car() sports_car =
SportsCar() # Demonstrate method calls
print("Vehicle:") vehicle.vehicle_info()
print("\nEngine:") engine.engine_type()
print("\nElectric Engine:")
electric_engine.engine_type()
electric_engine.electric_engine_info()
print("\nCar:") car.vehicle_info()
car.engine_type() car.electric_engine_info()
car.car_info() print("\nSports Car:")
sports_car.vehicle_info()
sports_car.engine_type()
sports_car.electric_engine_info()
sports_car.car_info()
sports_car.sports_car_info() #Demonstrate
MRO. print('\nMethod Resolution Order for
SportsCar:', SportsCar.__mro__)

```

Q6. Write a Python program to demonstrate the use of super() in inheritance.

```
class Parent: """Base class demonstrating
super().""" def __init__(self, name): """Initializes
the parent's name.""" self.name = name
print(f"Parent __init__ called with name:
{name}") def parent_method(self): """Parent
class method.""" print("Parent method called.")
class Child(Parent): """Child class inheriting from
Parent, demonstrating super().""" def
__init__(self, name, age): """Initializes the child's
name and age.""" super().__init__(name) # Call
parent's __init__ self.age = age print(f"Child
__init__ called with age: {age}") def
child_method(self): """Child class method,
calling parent method."""
super().parent_method() # Call parent's method
print("Child method called.") def
display_info(self): """Displays child and parent
information.""" print(f"Name: {self.name}, Age:
{self.age}") # Create an object of the Child class
child_obj = Child("Alice", 10) # Call child
methods, which also call parent methods
child_obj.child_method() child_obj.display_info()
```

```
#Demonstrate super() with multiple inheritance.
class Base1:
    def method(self):
        print("Base1 method")
class Base2:
    def method(self):
        print("Base2 method")
class Derived(Base1, Base2):
    def method(self):
        super().method()
#calls the method of the first parent in the MRO.
print("Derived method")
derived_object = Derived()
derived_object.method()
print(Derived.__mro__) #print the method resolution order.
```

8. Method Overloading and Overriding

Theory:

Q1. Method overloading: defining multiple methods with the same name but different parameters

Python, unlike some other programming languages like Java or C++, does not support traditional method overloading in the way you might expect. In those languages, you can define multiple methods with the same name within a class, as long as they have different parameter

lists (either in the number of parameters or their types).

However, Python's dynamic typing and flexible argument handling allow you to achieve similar functionality using default arguments, variable-length argument lists (*args and **kwargs), and type checking within a single method.

Here's how you can simulate method overloading in Python:

1. Using Default Arguments:

- You can define a single method with optional parameters that have default values.
- The method can then behave differently based on which parameters are provided.

Python

```
class Calculator:
```

```
    def add(self, a, b=0, c=0):
```

```
        if c != 0:
```

```
            return a + b + c
```



```
elif b != 0:
    return a + b
else:
    return a
```

```
calc = Calculator()
print(calc.add(5))    # Output: 5
print(calc.add(5, 10)) # Output: 15
print(calc.add(5, 10, 15)) # Output: 30
```

2. Using Variable-Length Arguments (*args and **kwargs):

- *args allows you to pass a variable number of positional arguments.
- **kwargs allows you to pass a variable number of keyword arguments.¹

[1. github.com](https://github.com)

github.com

Python

```
class MathOperations:
```

```
    def multiply(self, *args):
```

```
        result = 1
```

```
        for num in args:
```

```
            result *= num
```

```
        return result
```

```
math_op = MathOperations()
```

```
print(math_op.multiply(2, 3))    # Output: 6
```

```
print(math_op.multiply(2, 3, 4, 5)) # Output: 120
```

3. Using Type Checking:

- You can check the types of the arguments passed to a method and perform different actions based on the types.

Python

```
class DataProcessor:  
    def process(self, data):  
        if isinstance(data, int):  
            return data * 2  
        elif isinstance(data, str):  
            return data.upper()  
        else:  
            return "Unsupported data type"
```

```
dp = DataProcessor()  
print(dp.process(10))    # Output: 20  
print(dp.process("hello")) # Output: HELLO  
print(dp.process([1, 2, 3])) # Output:  
Unsupported data type
```

Important Considerations:

- **Readability:** While these techniques simulate overloading, they can sometimes make code less readable if not used carefully.
- **Maintainability:** If you have many variations of a method, it might be better to create separate methods with descriptive names.
- **Pythonic Approach:** The Pythonic way is often to use default arguments or `*args` and `**kwargs` when appropriate, rather than trying to strictly replicate traditional overloading.

Q2. Method overriding: redefining a parent class method in the child class

Method overriding is a fundamental concept in object-oriented programming (OOP) that allows a subclass (child class) to provide a specific implementation for a method that is already defined in its superclass (parent class). When a method in ¹ a subclass has the same name, same parameters, and same return type (or a compatible return type) as a method in its superclass, it overrides the superclass method.

[1. github.com](https://1.github.com)

github.com

Here's a breakdown of method overriding in Python:

Key Concepts:

- **Inheritance:** Method overriding is closely tied to inheritance. A subclass inherits methods from its superclass.
- **Redefinition:** The subclass redefines the method from the superclass to provide its own specialized implementation.
- **Polymorphism:** Method overriding is a form of polymorphism, which means "many forms." It allows objects of different classes to respond to the same method call in different ways.

Example:

Python

```
class Animal:
```

```
    def speak(self):
```

```
        print("Generic animal sound")
```

```
class Dog(Animal):
```

```
    def speak(self): # Method overriding
```

```
        print("Woof!")
```

```
class Cat(Animal):
```

```
    def speak(self): # Method overriding
```

```
        print("Meow!")
```

```
# Create objects
```

```
animal = Animal()
```

```
dog = Dog()
```

```
cat = Cat()
```

Call the speak method

animal.speak() # Output: Generic animal sound

dog.speak() # Output: Woof!

cat.speak() # Output: Meow!

Explanation:

1.class Animal::

- Defines a base class Animal with a speak() method that provides a generic animal sound.

2.class Dog(Animal): and class Cat(Animal)::

- Define subclasses Dog and Cat that inherit from Animal.
- Both subclasses override the speak() method to provide dog-specific ("Woof!") and cat-specific ("Meow!") sounds.

3.Object Creation and Method Calls:

- Objects of Animal, Dog, and Cat are created.

- When the speak() method is called on each object, the appropriate implementation is executed.
- For animal, the Animal class's speak() method is called.
- For dog, the Dog class's overridden speak() method is called.
- For cat, the Cat class's overridden speak() method is called.

Using super() with Overriding:

- You can use the super() function to call the overridden method from the superclass within the subclass's method.

Python

```
class Parent:
```

```
    def method(self):
```

```
        print("Parent method")
```

```
class Child(Parent):
```



```
def method(self):  
    super().method() # Call the parent's method  
    print("Child method")
```

```
child_obj = Child()  
child_obj.method()
```

Lab:

Q1. Write Python programs to demonstrate method overloading and method overriding.

Python

Method Overloading (Simulated)

```
class Calculator:  
    """Simulates method overloading using default  
    arguments."""
```

```
def add(self, a, b=0, c=0):  
    """Simulates adding two or three numbers."""
```

```
if c != 0:
    return a + b + c
elif b != 0:
    return a + b
else:
    return a
```

```
calc = Calculator()
print(calc.add(5))    # Output: 5
print(calc.add(5, 10)) # Output: 15
print(calc.add(5, 10, 15)) # Output: 30
```

Method Overriding

```
class Animal:
    """Base class for animals."""

    def speak(self):
```

```
"""Generic animal sound."""
```

```
print("Generic animal sound")
```

```
class Dog(Animal):
```

```
    """Subclass for dogs, overriding the speak  
    method."""
```

```
    def speak(self):
```

```
        """Dog-specific sound."""
```

```
        print("Woof!")
```

```
class Cat(Animal):
```

```
    """Subclass for cats, overriding the speak  
    method."""
```

```
    def speak(self):
```

```
        """Cat-specific sound."""
```

```
        print("Meow!")
```

```
# Create objects
```

```
animal = Animal()
```

```
dog = Dog()
```

```
cat = Cat()
```

```
# Call the speak method
```

```
animal.speak() # Output: Generic animal sound
```

```
dog.speak()   # Output: Woof!
```

```
cat.speak()   # Output: Meow!
```

```
# Demonstrate super() with overriding.
```

```
class Parent:
```

```
    def method(self):
```

```
        print("Parent method")
```

```
class Child(Parent):
```

```
    def method(self):
```

```
super().method()  
print("Child method")
```

```
child_obj = Child()  
child_obj.method()
```

Practical Examples:

Q1. Write a Python program to show method overloading.

```
class Calculator: """Simulates method  
overloading using default arguments."""  
def add(self, a, b=None, c=None): """Simulates  
adding two or three numbers."""  
if a is not None and b is not None and c is not None:  
    return a + b + c  
elif a is not None and b is not None:  
    return a + b  
elif a is not None:  
    return a  
else:  
    return 0  
# Handle case where no arguments are provided.  
def multiply(self, a, b=1, c=1): """Simulates  
multiplying two or three numbers."""  
return a * b * c  
def process_string(self, string, upper=False,  
reverse=False): """Simulates string processing
```

```
with optional arguments.""" if upper and reverse:
return string.upper()[::-1] elif upper: return
string.upper() elif reverse: return string[::-1] else:
return string # Create an object of the Calculator
class calc = Calculator() # Demonstrate the add
method print("Add:") print(calc.add(5))
print(calc.add(5, 10)) print(calc.add(5, 10, 15)) #
Demonstrate the multiply method
print("\nMultiply:") print(calc.multiply(2))
print(calc.multiply(2, 3)) print(calc.multiply(2, 3,
4)) # Demonstrate the string processing method
print("\nString processing:")
print(calc.process_string("hello"))
print(calc.process_string("hello", upper=True))
print(calc.process_string("hello", reverse=True))
print(calc.process_string("hello", upper=True,
reverse=True)) #Demonstrate the case where no
arguments are passed. print("\nAdd with no
arguments:") print(calc.add())
```

Q2. Write a Python program to show method overriding

```

class Animal: """Base class representing an
animal.""" def speak(self): """Generic animal
sound.""" print("Generic animal sound") def
move(self): """Generic animal movement."""
print("Animal moves") class Dog(Animal):
"""Subclass representing a dog, overriding
methods from Animal.""" def speak(self): """Dog-
specific sound.""" print("Woof!") def move(self):
"""Dog-specific movement.""" print("Dog runs")
class Cat(Animal): """Subclass representing a
cat, overriding methods from Animal.""" def
speak(self): """Cat-specific sound."""
print("Meow!") def move(self): """Cat-specific
movement.""" print("Cat walks") # Create objects
animal = Animal() dog = Dog() cat = Cat() # Call
the speak and move methods print("Animal:")
animal.speak() animal.move() print("\nDog:")
dog.speak() dog.move() print("\nCat:")
cat.speak() cat.move() # Demonstrating super()
with method overriding. class Parent: def
method(self): print("Parent method called")
class Child(Parent): def method(self):

```

```
super().method() # Call the parent's method  
print("Child method called") child_object =  
Child() child_object.method()
```

9.SQLite3 and PyMySQL (Database Connectors)

Theory:

Q1.Introduction to SQLite3 and PyMySQL for database connectivity.

SQLite3 and PyMySQL are Python libraries used for database connectivity, but they cater to different database systems and use cases.

Here's an introduction to each:

SQLite3:

. What it is:

- SQLite3 is a built-in Python module that provides an interface to the SQLite database engine.
- SQLite is a self-contained, serverless, zero-configuration, transactional SQL database engine.

- It stores the entire database in a single file on your local machine.

- **Key features:**

- **Serverless:** No separate database server is required.
- **File-based:** Databases are stored in single files, making them easy to distribute.
- **Lightweight:** Ideal for small to medium-sized applications, embedded systems, and development/testing.
- **Transactional:** Supports ACID transactions.
- **Built-in:** Part of the Python standard library, so no external installation is needed.

- **Use cases:**

- Desktop applications.
- Mobile applications.
- Embedded systems.

- Prototyping and testing.
- Simple web applications with low traffic.

• **Example (basic):**

Python

```
import sqlite3
```

```
# Connect to a database (or create it if it doesn't  
exist)
```

```
conn = sqlite3.connect('my_database.db')
```

```
# Create a cursor object
```

```
cursor = conn.cursor()
```

```
# Execute an SQL command
```

```
cursor.execute("""CREATE TABLE IF NOT EXISTS  
users (id INTEGER PRIMARY KEY, name TEXT,  
age INTEGER)""")
```

Insert data

```
cursor.execute("INSERT INTO users (name, age)  
VALUES ('Alice', 30)")  
  
conn.commit()
```

Retrieve data

```
cursor.execute("SELECT * FROM users")  
  
rows = cursor.fetchall()  
  
print(rows)
```

Close the connection

```
conn.close()
```

PyMySQL:

. What it is:

- PyMySQL is a pure-Python MySQL client library.
- It allows you to connect to and interact with MySQL databases.

- MySQL is a powerful, widely used, open-source relational database management system (RDBMS).

- **Key features:**

- **MySQL connectivity:** Enables Python applications to communicate with MySQL servers.
- **Pure Python:** Written in Python, making it cross-platform.
- **Comprehensive support:** Supports most MySQL features.
- **Widely used:** Popular for web applications and other applications that require a robust RDBMS.

- **Use cases:**

- Web applications (e.g., Django, Flask).
- Large-scale applications.
- Applications requiring a centralized database server.

- Any application using a MySQL database.

- **Example (basic):**

Python

```
import pymysql
```

```
# Connect to a MySQL database
```

```
conn = pymysql.connect(host='localhost',  
user='your_user', password='your_password',  
database='your_database')
```

```
# Create a cursor object
```

```
cursor = conn.cursor()
```

```
# Execute an SQL command
```

```
cursor.execute("SELECT VERSION()")
```

```
version = cursor.fetchone()
```

```
print(f"MySQL version: {version[0]}")
```

```
# Close the connection
```

```
conn.close()
```

Key Differences:

- **Database system:** SQLite3 is for SQLite databases, while PyMySQL is for MySQL databases.
- **Server requirements:** SQLite3 is serverless, while MySQL requires a separate server.
- **Storage:** SQLite3 stores data in a single file, while MySQL stores data in a server-managed environment.
- **Use cases:** SQLite3 is for lightweight applications, while PyMySQL is for more robust, server-based applications.
- **Installation:** SQLite3 is built-in, while PyMySQL requires external installation (pip install pymysql).
- **Concurrency:** SQLite3 has limitations with high concurrency, while MySQL is designed for concurrent access.

Q2. Creating and executing SQL queries from Python using these connectors.

Let's demonstrate how to create and execute SQL queries from Python using both sqlite3 and pymysql connectors.

1. Using sqlite3:

Python

```
import sqlite3
```

```
def execute_sqlite_queries(db_filename):
```

```
    """Executes SQL queries on an SQLite
    database."""
```

```
    try:
```

```
        # Connect to the SQLite database
```

```
        conn = sqlite3.connect(db_filename)
```

```
        cursor = conn.cursor()
```

```
# Create a table (if it doesn't exist)

cursor.execute("""CREATE TABLE IF NOT
EXISTS employees (

    id INTEGER PRIMARY KEY,

    name TEXT,

    department TEXT,

    salary REAL

)""")
```

```
# Insert data

cursor.execute("INSERT INTO employees
(name, department, salary) VALUES ('Alice',
'Sales', 50000)")

cursor.execute("INSERT INTO employees
(name, department, salary) VALUES ('Bob',
'Engineering', 75000)")

conn.commit()
```

```
# Retrieve data
```



```
    cursor.execute("SELECT * FROM  
employees")
```

```
    rows = cursor.fetchall()
```

```
    print("Employees:")
```

```
    for row in rows:
```

```
        print(row)
```

```
# Update data
```

```
    cursor.execute("UPDATE employees SET  
salary = 80000 WHERE name = 'Bob'")
```

```
    conn.commit()
```

```
# Retrieve updated data
```

```
    cursor.execute("SELECT * FROM  
employees")
```

```
    rows = cursor.fetchall()
```

```
    print("\nUpdated Employees:")
```

```
    for row in rows:
```

```
print(row)
```

```
# Delete data
```

```
cursor.execute("DELETE FROM employees  
WHERE name = 'Alice'")
```

```
conn.commit()
```

```
# Retrieve remaining data
```

```
cursor.execute("SELECT * FROM  
employees")
```

```
rows = cursor.fetchall()
```

```
print("\nRemaining Employees:")
```

```
for row in rows:
```

```
    print(row)
```

```
except sqlite3.Error as e:
```

```
    print(f"SQLite error: {e}")
```

finally:

if conn:

conn.close()

Example usage

execute_sqlite_queries('employees.db')

2. Using pymysql:

Python

import pymysql

def execute_mysql_queries(host, user,
password, database):

"""Executes SQL queries on a MySQL
database."""

try:

Connect to the MySQL database

```
conn = pymysql.connect(host=host,
user=user, password=password,
database=database)

cursor = conn.cursor()

# Create a table (if it doesn't exist)

cursor.execute("""CREATE TABLE IF NOT
EXISTS products (

            id INT AUTO_INCREMENT
PRIMARY KEY,

            name VARCHAR(255),

            price DECIMAL(10, 2)

)""")

# Insert data

cursor.execute("INSERT INTO products
(name, price) VALUES ('Laptop', 1200.50)")

cursor.execute("INSERT INTO products
(name, price) VALUES ('Mouse', 25.99)")
```

```
conn.commit()
```

```
# Retrieve data
```

```
cursor.execute("SELECT * FROM products")
```

```
rows = cursor.fetchall()
```

```
print("Products:")
```

```
for row in rows:
```

```
    print(row)
```

```
# Update data
```

```
cursor.execute("UPDATE products SET price  
= 30.00 WHERE name = 'Mouse'")
```

```
conn.commit()
```

```
# Retrieve updated data
```

```
cursor.execute("SELECT * FROM products")
```

```
rows = cursor.fetchall()
```

```
print("\nUpdated Products:")
```

```
for row in rows:
```

```
    print(row)
```

```
# Delete data
```

```
    cursor.execute("DELETE FROM products  
WHERE name = 'Laptop'")
```

```
    conn.commit()
```

```
# Retrieve remaining data
```

```
    cursor.execute("SELECT * FROM products")
```

```
    rows = cursor.fetchall()
```

```
    print("\nRemaining Products:")
```

```
    for row in rows:
```

```
        print(row)
```

```
except pymysql.MySQLError as e:
```

```
    print(f"MySQL error: {e}")
```

finally:

if conn:

conn.close()

Example usage (replace with your MySQL credentials)

```
execute_mysql_queries(host='localhost',  
user='your_user', password='your_password',  
database='your_database')
```

Key Points:

. Connections:

- `sqlite3.connect()` creates a connection to an SQLite database file.
- `pymysql.connect()` creates a connection to a MySQL server.

. Cursors:

- `cursor()` objects are used to execute SQL queries.

. Executing Queries:

- `cursor.execute()` executes SQL statements.

- **Retrieving Data:**

- `cursor.fetchall()` retrieves all rows from a query result.
- `cursor.fetchone()` retrieves a single row.

- **Committing Changes:**

- `conn.commit()` saves changes to the database.

- **Closing Connections:**

- `conn.close()` closes the database connection.

- **Error Handling:**

- `try...except...finally` blocks are used to handle potential database errors and ensure that connections are closed properly.

- **Security:**

- In real-world applications, avoid embedding sensitive credentials directly in your code. Use environment variables or configuration files.
- Sanitize user inputs to prevent SQL injection vulnerabilities.

- **Placeholders:**

- Use placeholders (?, %s for MySQL) to pass parameters to SQL queries safely. This prevents SQL injection.
- Example: `cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ('John', 25))`

Lab:

Q1. Write a Python program to connect to an SQLite3 database, create a table, insert data, and fetch data.

```
import sqlite3
def interact_with_sqlite_db(db_filename):
    """Connects to an SQLite3 database, creates a
```

```
table, inserts data, and fetches data.""" try: #
Connect to the SQLite3 database (or create it if it
doesn't exist) conn =
sqlite3.connect(db_filename) cursor =
conn.cursor() # Create a table named 'students'
(if it doesn't exist) cursor.execute("""CREATE
TABLE IF NOT EXISTS students ( id INTEGER
PRIMARY KEY, name TEXT, age INTEGER, grade
TEXT )""") # Insert data into the 'students' table
cursor.execute("INSERT INTO students (name,
age, grade) VALUES ('Alice', 18, 'A')")
cursor.execute("INSERT INTO students (name,
age, grade) VALUES ('Bob', 19, 'B')")
cursor.execute("INSERT INTO students (name,
age, grade) VALUES ('Charlie', 20, 'C')") # Commit
the changes conn.commit() # Fetch all data from
the 'students' table cursor.execute("SELECT *
FROM students") rows = cursor.fetchall() # Print
the fetched data print("Students Data:") for row
in rows: print(row) #Fetch data based on a
condition cursor.execute("SELECT name, grade
FROM students WHERE age > 18") rows_filtered
```

```
= cursor.fetchall() print("\nStudents older than  
18:") for row in rows_filtered: print(row) except  
sqlite3.Error as e: print(f"SQLite error: {e}")  
finally: # Close the connection (important!) if  
conn: conn.close() # Example usage  
interact_with_sqlite_db('students.db')
```

Practical Examples:

Q1. Write a Python program to create a database and a table using SQLite3.

```
import sqlite3 def  
create_database_and_table(db_filename,  
table_name, columns): """ Creates an SQLite3  
database and a table within it. Args: db_filename  
(str): The name of the database file. table_name  
(str): The name of the table to create. columns  
(list): A list of tuples, where each tuple  
represents a column (column_name, data_type).  
""" try: # Connect to the SQLite3 database (or  
create it if it doesn't exist) conn =  
sqlite3.connect(db_filename) cursor =  
conn.cursor() # Construct the CREATE TABLE
```

```

SQL statement column_definitions = ",
".join([f"{name} {data_type}" for name, data_type
in columns]) create_table_sql = f"CREATE TABLE
IF NOT EXISTS {table_name}
({column_definitions})" # Execute the CREATE
TABLE statement
cursor.execute(create_table_sql) # Commit the
changes and close the connection
conn.commit() print(f"Database '{db_filename}'
and table '{table_name}' created successfully.")
except sqlite3.Error as e: print(f"SQLite error:
{e}") finally: if conn: conn.close() # Example
usage: db_file = "my_database.db" table =
"products" columns = [ ("id", "INTEGER PRIMARY
KEY AUTOINCREMENT"), ("name", "TEXT"),
("price", "REAL"), ("quantity", "INTEGER") ]
create_database_and_table(db_file, table,
columns)

```

Q2. Write a Python program to insert data into an SQLite3 database and fetch it.

```

import sqlite3
def insert_and_fetch_data(db_filename,

```

table_name, data): """ Inserts data into an SQLite3 database table and fetches it. Args: db_filename (str): The name of the database file. table_name (str): The name of the table to insert data into. data (list): A list of tuples, where each tuple represents a row of data. """ try: # Connect to the SQLite3 database (or create it if it doesn't exist) conn = sqlite3.connect(db_filename) cursor = conn.cursor() # Insert data into the table placeholders = ", ".join(["?" for _ in data[0]]) # Create placeholders (?, ?, ...) insert_sql = f"INSERT INTO {table_name} VALUES ({placeholders})" cursor.executemany(insert_sql, data) #use executemany for bulk inserts. # Commit the changes conn.commit() print("Data inserted successfully.") # Fetch all data from the table cursor.execute(f"SELECT * FROM {table_name}") rows = cursor.fetchall() # Print the fetched data print("\nFetched data:") for row in rows: print(row) except sqlite3.Error as e: print(f"SQLite error: {e}") finally: if conn: conn.close() # Example usage: db_file =

```
"my_database.db" table = "employees"
employee_data = [ ("Alice", 30, "Sales"), ("Bob",
25, "Engineering"), ("Charlie", 35, "Marketing") ] #
Create the table if it does not exist. conn =
sqlite3.connect(db_file) cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS
employees (name TEXT, age INTEGER,
department TEXT)""") conn.commit() conn.close()
insert_and_fetch_data(db_file, table,
employee_data)
```

10. Search and Match Functions

Theory:

Q1. Using re.search() and re.match() functions in Python's re module for pattern matching.

The re module in Python provides regular expression matching operations. re.search() and re.match() are two fundamental functions within this module, but they differ in how they approach pattern matching within a string.

1. re.search(pattern, string):

- Purpose:**

- Scans the entire string for the first location where the regular expression pattern produces a match.

- **Behavior:**

- If a match is found anywhere within the string, it returns a match object.
- If no match is found, it returns None.

- **Example:**

Python

```
import re
```

```
text = "The quick brown fox jumps over the lazy  
dog"
```

```
pattern = r"brown"
```

```
match = re.search(pattern, text)
```

```
if match:
```

```
print("Match found:", match.group()) # Output:  
Match found: brown
```

```
print("Start index:", match.start()) #Output: 10
```

```
print("End index:", match.end()) #Output: 15
```

```
else:
```

```
print("Match not found")
```

```
text2 = "fox"
```

```
match2 = re.search(pattern, text2)
```

```
if match2:
```

```
print("Match found:", match2.group())
```

```
else:
```

```
print("Match not found") #Output: Match not  
found.
```

2. re.match(pattern, string):

- . Purpose:**

- Attempts to match the regular expression pattern at the *beginning* of the string.

- **Behavior:**

- If a match is found at the start of the string, it returns a match object.
- If no match is found at the beginning, it returns None.

- **Example:**

Python

```
import re
```

```
text = "The quick brown fox"
```

```
pattern = r"The"
```

```
match = re.match(pattern, text)
```

```
if match:
```

```
print("Match found:", match.group()) #Output:  
Match found: The
```

```
print("Start index:", match.start()) #output: 0
```

```
print("End index:", match.end()) #output: 3
```

```
else:
```

```
print("Match not found")
```

```
text2 = "quick brown fox"
```

```
match2 = re.match(pattern, text2)
```

```
if match2:
```

```
print("Match found:", match2.group())
```

```
else:
```

```
print("Match not found") #Output: Match not  
found
```

Key Differences Summarized:

- Search Location:**

- `re.search()`: Searches the entire string for a match.
- `re.match()`: Matches only at the beginning of the string.

- **Return Behavior:**

- Both return a match object if a match is found, or `None` if not.

- **Use Cases:**

- `re.search()`: Use when you need to find a pattern anywhere within a string.
- `re.match()`: Use when you need to check if a string starts with a specific pattern.

Example with ^ and \$:

Python

```
import re
```

```
text = "start middle end"
```

```
pattern_start = r"^start" #match start of string.
```

```
pattern_end = r"end$" #match end of string.
```

```
pattern_middle = r"middle"
```

```
match_start = re.search(pattern_start, text)
```

```
match_end = re.search(pattern_end, text)
```

```
match_middle = re.search(pattern_middle, text)
```

```
if match_start:
```

```
    print("Start match:", match_start.group())
```

```
#output: start match: start.
```

```
if match_end:
```

```
    print("End match:", match_end.group())
```

```
#output: End match: end.
```

```
if match_middle:
```

```
    print("Middle match:", match_middle.group())
```

```
#output: Middle match: middle.
```

Q2. Difference between search and match.

The key difference between `re.search()` and `re.match()` in Python's `re` module lies in where they look for a match within a string:

- **`re.match(pattern, string):`**

- This function checks for a match **only at the beginning** of the string.
- If the pattern matches at the very start, it returns a match object.
- If the pattern doesn't match at the start, it returns `None`.
- Essentially, it asks, "Does this string *start with* this pattern?"

- **`re.search(pattern, string):`**

- This function scans the **entire string** for the first location where the pattern matches.
- If a match is found anywhere within the string, it returns a match object.
- If no match is found, it returns `None`.

- Essentially, it asks, "Does this string *contain* this pattern anywhere?"

Here's a concise comparison:

Feature	re.match()	re.search()
Search Location	Beginning of the string only	Anywhere in the string
Return Value	Match object (if match at start), None otherwise	Match object (if match found), None otherwise
Use Case	Checking if a string <i>starts with</i> a pattern	Finding a pattern <i>anywhere within</i> a string

Export to Sheets

Example to illustrate:

Python

```
import re
```

```
text = "The quick brown fox"
```

```
pattern = r"quick"
```

```
match_result = re.match(pattern, text)
search_result = re.search(pattern, text)
```

```
if match_result:
    print("match found at the begining")
else:
    print("match not found at the begining")
```

```
if search_result:
    print("search found a match")
else:
    print("search did not find a match")
```

#output:

#match not found at the begining

#search found a match

In this example:

- `re.match()` fails because "quick" is not at the beginning of the string.
- `re.search()` succeeds because "quick" is found within the string.

Lab:

Q1. Write a Python program to search for a word in a string using `re.search()`.

```
import re
def find_word_in_string(text, word): """
Searches for a word within a string using
re.search(). Args: text (str): The string to search
within. word (str): The word to search for.
Returns: str: A message indicating whether the
word was found, or None if an error occurred. """
try: pattern = r"\b" + re.escape(word) + r"\b" #
Use word boundaries (\b) and escape special
characters. match = re.search(pattern, text) if
match: return f"Word '{word}' found at position
{match.start()} to {match.end()}." else: return
f"Word '{word}' not found." except re.error as e:
return f"Regular expression error: {e}" # Example
Usage: my_string = "This is a sample string, with
```


the word 'sample' in it. sample also appears at the end." search_word = "sample" result = find_word_in_string(my_string, search_word) print(result) search_word = "not_present" result = find_word_in_string(my_string, search_word) print(result) search_word = "." #search for a special character. result = find_word_in_string(my_string, search_word) print(result) search_word = "sample." #search for a word with a special character. result = find_word_in_string(my_string, search_word) print(result)

Q2. Write a Python program to match a word in a string using re.match().

```
import re
def match_word_at_beginning(text, word):
    """ Matches a word at the beginning of a string using re.match().
    Args: text (str): The string to match against.
    word (str): The word to match at the beginning.
    Returns: str: A message indicating whether the word was matched, or None if an error occurred. """
    try:
        pattern = r"\b" + re.escape(word) + r"\b"
        # Use word boundaries
```

```

and escape special characters. match =
re.match(pattern, text) if match: return f"Word
'{word}' matched at the beginning. Position:
{match.start()} to {match.end()}." else: return
f"Word '{word}' not matched at the beginning."
except re.error as e: return f"Regular expression
error: {e}" # Example Usage: my_string = "sample
string with sample word" match_word =
"sample" result =
match_word_at_beginning(my_string,
match_word) print(result) my_string2 = "string
with sample word" result2 =
match_word_at_beginning(my_string2,
match_word) print(result2) my_string3 =
"sample.word and more" result3 =
match_word_at_beginning(my_string3,"sample.w
ord") print(result3) my_string4 = ".sample"
result4 =
match_word_at_beginning(my_string4,".sample")
print(result4)

```

Practical Examples:

Q1. Write a Python program to search for a word in a string using re.search().

```
import re
def search_word_in_string(text, word):
    """ Searches for a word in a string using
    re.search(). Args: text (str): The string to search
    in. word (str): The word to search for. Returns: str:
    A message indicating whether the word was
    found, or None if an error occurred. """
    try: #
        Construct the regular expression pattern with
        word boundaries. pattern = r"\b" +
        re.escape(word) + r"\b" # Search for the word in
        the string. match = re.search(pattern, text)
        if match: return f"Word '{word}' found at position
        {match.start()} to {match.end()}."
        else: return
        f"Word '{word}' not found."
    except re.error as e:
        return f"Regular expression error: {e}"
# Example
usage: my_string = "This is a sample string, with
the word sample in it. samples is not a match."
search_word = "sample"
result =
search_word_in_string(my_string, search_word)
print(result)
search_word = "not_found"
result =
search_word_in_string(my_string, search_word)
```

```

print(result) search_word = "." #search for a
special character. result =
search_word_in_string(my_string, search_word)
print(result) search_word = "sample." #search for
a word with a special character. result =
search_word_in_string(my_string, search_word)
print(result) search_word = "is" #search for a
common word. result =
search_word_in_string(my_string, search_word)
print(result)

```

Q2. Write a Python program to match a word in a string using re.match().

```

import re
def match_word_at_beginning(text, word):
    """ Matches a word at the beginning of a
    string using re.match(). Args: text (str): The string
    to match against. word (str): The word to match
    at the beginning. Returns: str: A message
    indicating whether the word was matched, or
    None if an error occurred. """
    try: # Construct the
    regular expression pattern with word boundaries.
    pattern = r"\b" + re.escape(word) + r"\b" # Match
    the word at the beginning of the string. match =

```

```
re.match(pattern, text) if match: return f"Word
'{word}' matched at the beginning. Position:
{match.start()} to {match.end()}." else: return
f"Word '{word}' not matched at the beginning."
except re.error as e: return f"Regular expression
error: {e}" # Example Usage: my_string = "sample
string with sample word" match_word =
"sample" result =
match_word_at_beginning(my_string,
match_word) print(result) my_string2 = "string
with sample word" match_word = "sample"
result2 = match_word_at_beginning(my_string2,
match_word) print(result2) my_string3 =
"sample.word and more" match_word =
"sample.word" result3 =
match_word_at_beginning(my_string3,
match_word) print(result3) my_string4 =
".sample and more" match_word = ".sample"
result4 = match_word_at_beginning(my_string4,
match_word) print(result4) my_string5 =
"sample123 test" match_word = "sample123"
result5 = match_word_at_beginning(my_string5,
```

```
match_word) print(result5) my_string6 =  
"123sample test" match_word = "123sample"  
result6 = match_word_at_beginning(my_string6,  
match_word) print(result6)
```